

BUILD • LAUNCH • SOAR

Advanced Aerial Robotics with PX4 and ROS: Custom Flight Modes and Beyond

github.com/Dronecode/roscon-25-workshop

Beniamino Pozzan
beniamino.pozzan@gmail.com

Patrik Pordi
pordipatrik@gmail.com

Alexis Guijarro
totonzx@gmail.com

Pre Workshop Survey





whoami

Beniamino Pozzan
Senior Robotics Engineer
Archangel Autonomy

- 5+ years in GNC for aerial robotics
- PX4 maintainer (ROS 2 integration)
- 3 years in GNSS-denied navigation
- PhD in Control System Engineering



whoami

Patrik Pordi
Robotics & Drone Engineer
University of Southern Denmark

- ROS2 teaching experience
- 3 years of PX4 experience
- PX4 driver development for ARK
- PhD fellow in nature conservation project



whoami

Alexis Guijarro
Lead Software Engineer
3DR

- 9 years in OSS for aerial robotics
- Co-Lead ROS Aerial Robotics CWG
- PX4 Board Maintainer for 3DR/mRo
- MS in CS / Mechatronics Eng

Day overview

- **08:00** – Introduction & Environment Setup
- **09:30** – PX4 ROS 2 Interface Library, Custom Flight Modes
- **11:00** – Mode Executors and Multi-Vehicle Control
- **12:00** – Lunch Break
- **13:00** – Advanced Control & Perception Applications
- **15:30** – From Simulation to Hardware & Architecture Patterns
- **16:30** – Q&A, Open Lab & Individual Consultations
- **20:00** – Aerial Robotics Meetup, sponsored by Dronecode

Aerial Robotics Meetup, tonight!

Tonight, 8:00 to 11:00 PM
CRAFT Beer Market, 1 Adelaide St E (short walk from the venue)
Sponsored by Dronecode. **Food and drinks are on us.**

Talk shop on control architectures, state estimation, and open source flight stacks with the people who actually maintain them.



Prerequisites - environment setup - architecture overview

In this section you will

- Understand the requirements for running the exercise
- Set up the simulation environment
- Fly a drone in Gazebo without ROS 2
- Attach ROS 2 to the simulation
- Echo PX4 topics in ROS 2
- Visualize PX4 data with Foxglove

Environment Setup

REQUIREMENTS:

- Install Foxglove `https://foxglove.dev/download`
- Download and install QGC `https://qgroundcontrol.com`
- Clone PX4 repo: https://docs.px4.io/main/en/sim_gazebo_gz/#installation-ubuntu-linux

```
git clone -b release/1.18 https://github.com/PX4/PX4-Autopilot.git
```

- Prepare workspace and clone workshop repo https://github.com/Dronecode/roscon-25-workshop :

```
mkdir -p ~/workspaces/px4_roscon26_ws/src
cd ~/workspaces/px4_roscon26_ws/src
git clone git@github.com:Dronecode/roscon-25-workshop.git px4-roscon-workshop
git clone https://github.com/Dronecode/roscon-25-workshop.git px4-roscon-workshop
cd ~/workspaces/px4_roscon26_ws
vcs import src < ./src/px4-roscon-workshop/jazzy.repos
```

Environment Setup, dockerless, Ubuntu 22.04/24.04, WSL 2

- Follow PX4 simulation setup instructions https://docs.px4.io/main/en/sim_gazebo_gz/#installation-ubuntu-linux

  To setup PX4 SITL build toolchain and download Gazebo Harmonic

  Verify it by starting a simple simulation over Gazebo, no ROS 2 involved:

  ```
  cd ~/PX4-Autopilot
  make px4_sitl gz_x500
  ```

- Install ROS 2 (Jazzy or Humble depending on OS version)

  https://docs.ros.org/en/jazzy/Installation/Ubuntu-Install-Debs.html. Pick `ros-jazzy-desktop`

  https://docs.ros.org/en/humble/Installation/Ubuntu-Install-Debs.html. Pick `ros-humble-desktop`

- Install ROS 2 developer tools: `sudo apt install ros-dev-tools`
- Install dependencies:

  ```
  cd ~/workspaces/px4_roscon26_ws
  source /opt/ros/jazzy/setup.bash # source /opt/ros/humble/setup.bash
  rosdep install -i --from-paths src --skip-keys OpenCV
  ```

Environment Setup, docker

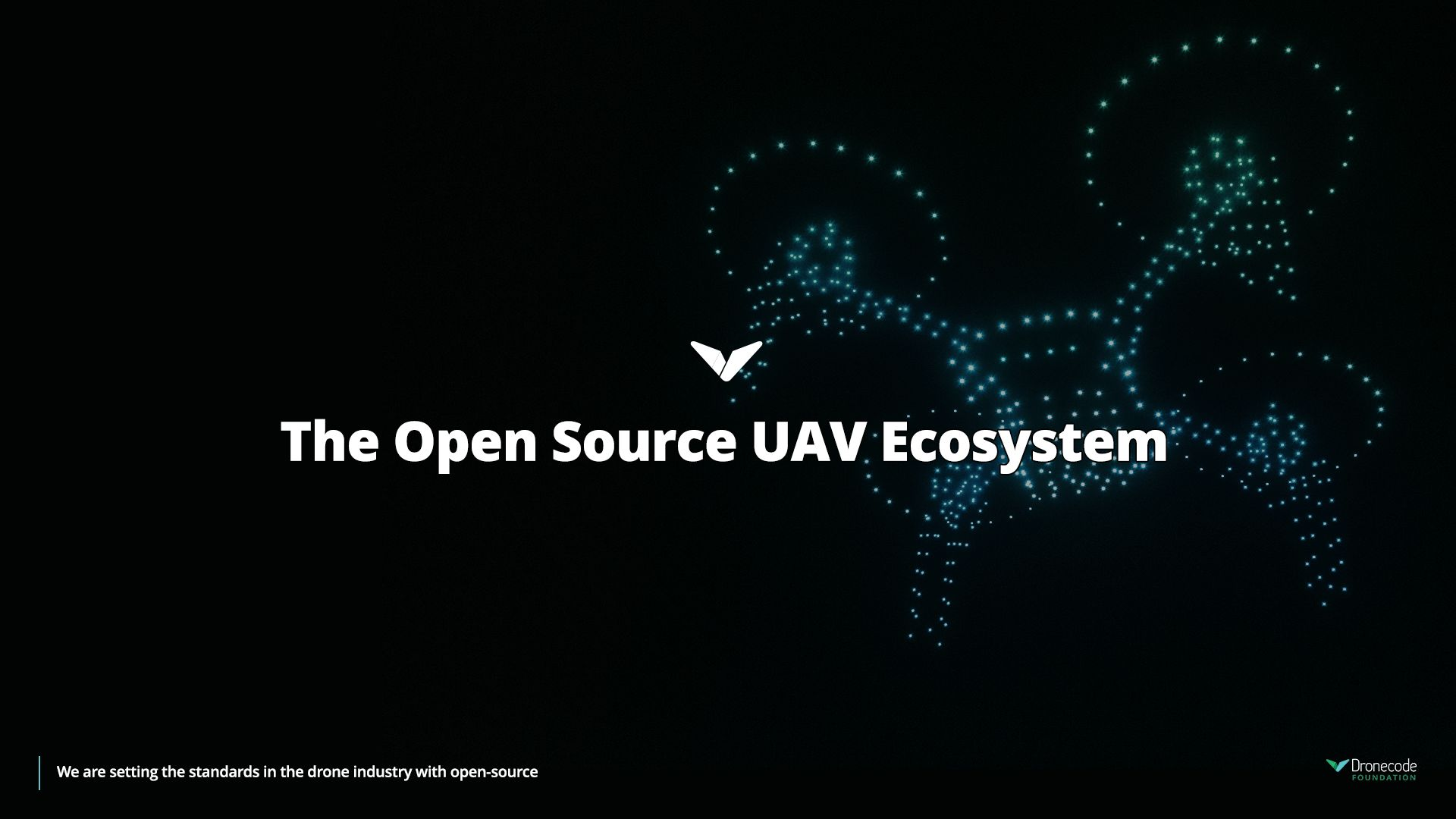
- Retrieve `px4io/px4-dev-ros2-gazebo:main-jazzy` docker image or build it yourself
  https://github.com/PX4/PX4-Autopilot/pull/28730
- Start the container

```
cd ~/workspaces/px4_roscon26_ws/src/ px4-roscon-workshop
./docker/docker_run.sh ~/PX4-Autopilot
```

- Set `HEADLESS=1` if you don't want GUI to run
- To leverage GPU acceleration, if available, refer to https://docs.px4.io/main/en/simulation/gazebo_container_gui
- From inside the container, install the workspace dependencies and build the workspace

```
apt update
rosdep install -y -i --from-paths src --skip-keys OpenCV
source /opt/ros/jazzy/setup.bash
colcon build --symlink-install
```


Dronecode
FOUNDATION

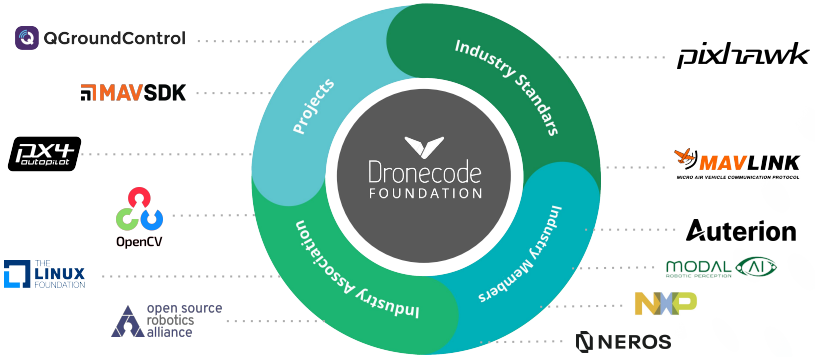


The Open Source UAV Ecosystem

We are setting the standards in the drone industry with open-source

Dronecode FOUNDATION

Neutral standards body for the drone industry since 2014



25 Years of being the place where competitors collaborate.

Members trust us with IP, governance, and brand. Maintainers trust us not to favor one company. Regulators trust our specs. That trust took 25 years, and is structurally hard for a new entity to replicate.

Global community of 15,000+ contributors

Production ready used by over 1 million drones

Convener, host, standards body, operator.

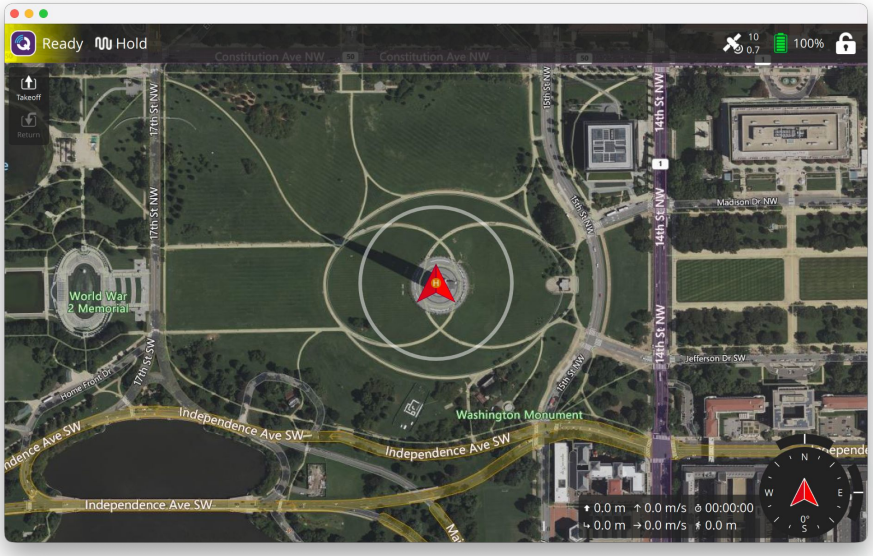
THE LINUX FOUNDATION

Dronecode FOUNDATION



Lifting the Industry

Open Source Autonomy Firmware and Compute Platforms



Lifting the Industry

Open Source Mission Planning and Telemetry



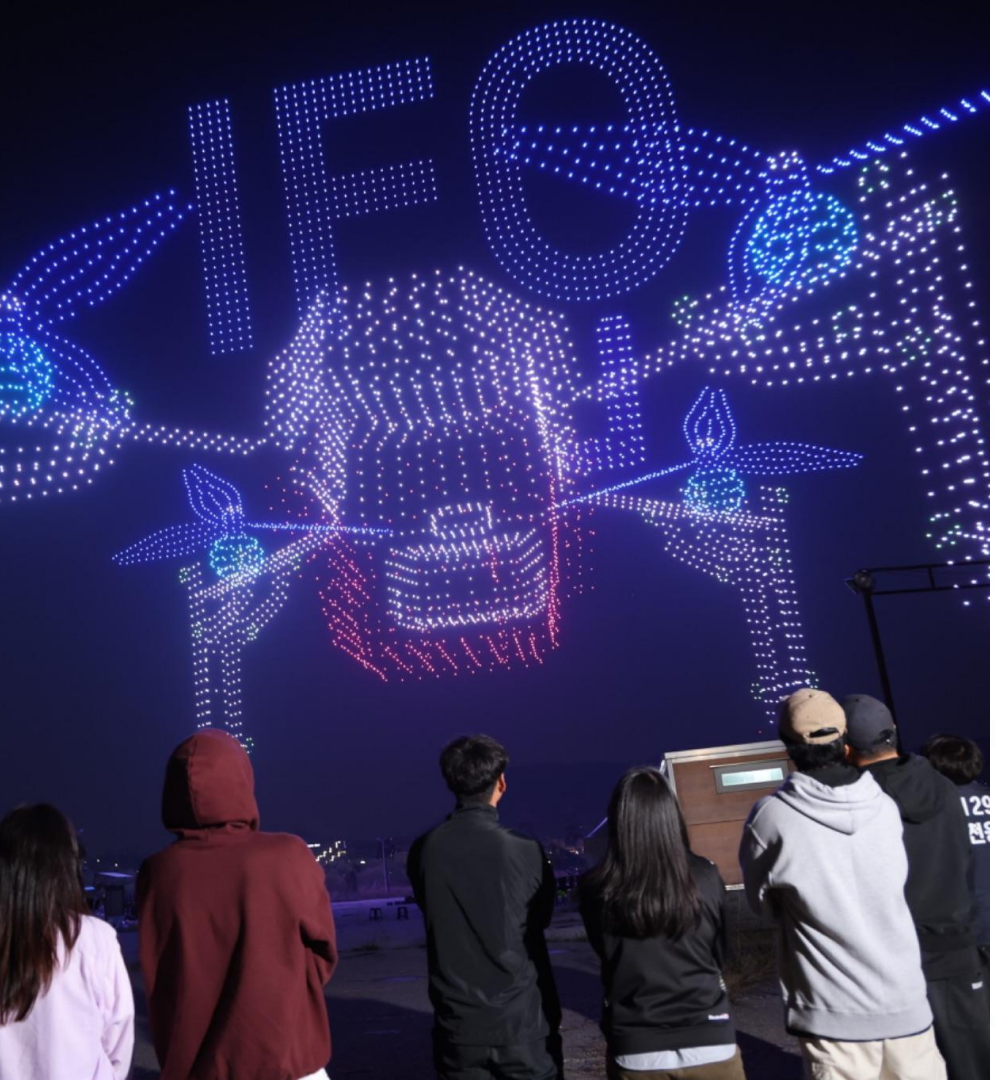
Lifting the Industry

Open Source Development Platforms



Lifting the Industry

Limitless vehicle type support from traditional Quadcopters to highly reliable VTOLs of all sizes



Lifting the Industry

Guinness record breaking drone swarms



Lifting the Industry

Battlefield tested drone swarms

Member Community

Dronecode FOUNDATION

Platinum

Auterion

DELTAQUAD

UVify

Gold

ST

Member Community

Dronecode FOUNDATION

Silver
































































Putting things into perspective

Lifetime Achievements

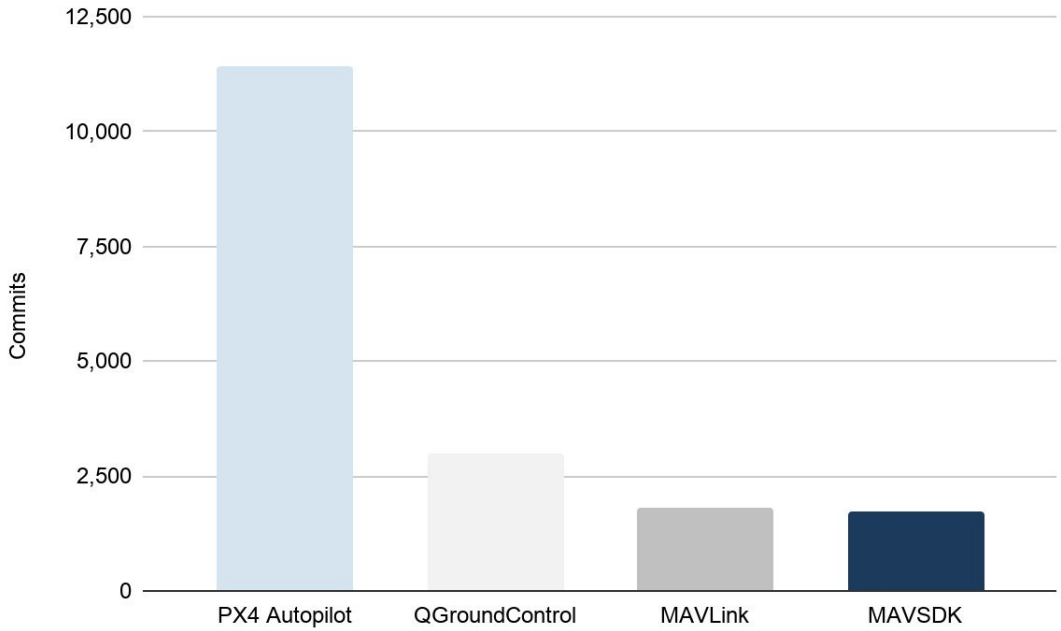
- $800M Project Value
- +60M Total Lines of Code
- +15K Total Unique Contributors



Data Provided by

LFX | Insights

Contribution Distribution by Project (2025)



Numbers from 2025

- +17,900 Commits
- +3,300 PRs submitted
- +2,800 PRs merged
- +650 Contributors
- +99 Organizations

2025 Annual Report: Year In Review

Read the full report for contribution metrics, project information, membership roster, and event reports

The Dronecode Stack











The Dronecode Stack

Autonomy Firmware. **PX4 Autopilot.**
The flight stack. Every domain: air, ground, water, and space

Autonomy Hardware. Pixhawk Open Standards.
Open hardware standards. 100-plus boards, 40-plus manufacturers.

Telemetry Protocols. MAVLink.
The messaging protocol. The lingua franca of uncrewed systems.

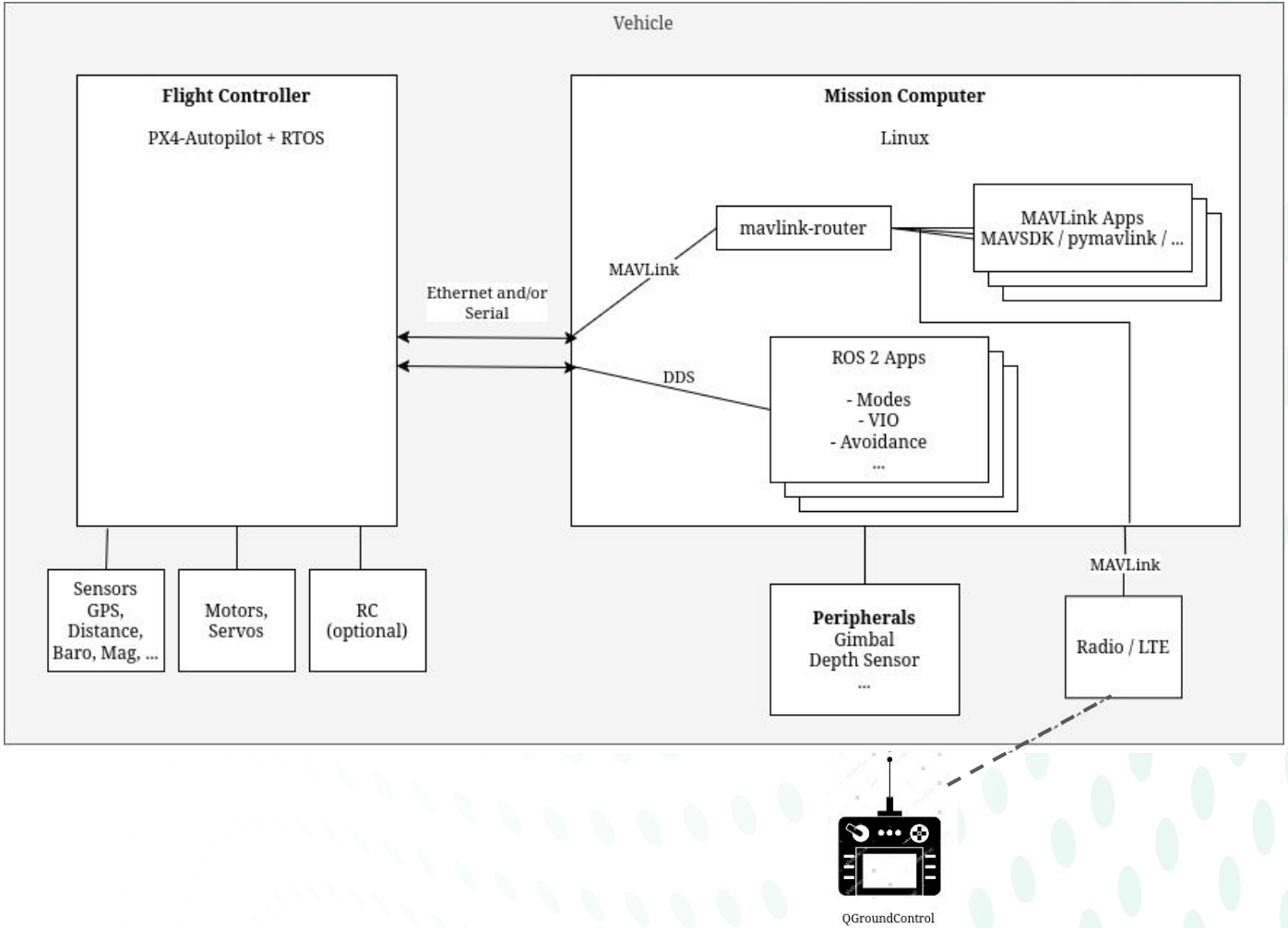
Telemetry SDK. MAVSDK.
The library developers use to talk to MAVLink. Multiple languages.

Mission Planner. QGroundControl.
The cross-platform ground control station. Desktop and mobile

PX4 autopilot pixhawk® MAVLINK MICRO AIR VEHICLE COMMUNICATION PROTOCOL MAVSDK GroundControl

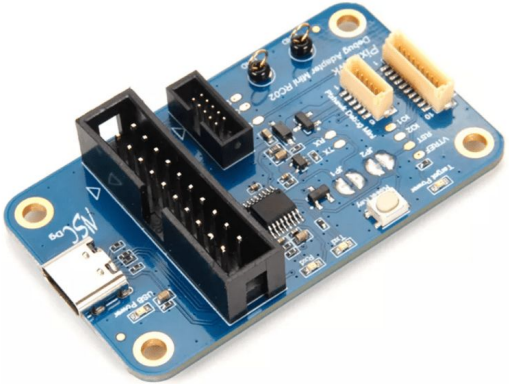
The Stack at a Glance

- Pixhawk
- PX4 Autopilot
- MAVLink
- MAVSDK
- QGroundControl
- ROS 2

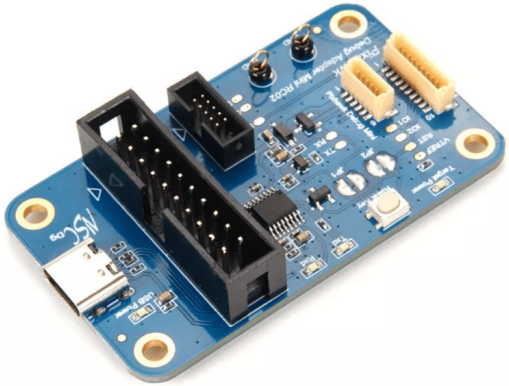


pixhawk®





Pixhawk Project: Autonomy Hardware



Open Hardware & Open Standards

Started as a flight controller open hardware project with the first versions of Pixhawk's

Evolved into an Open Standard for

- Flight controllers
- Payloads (Gimbals)
- Smart Batteries
- Connectors
- Debuggers





pixhawk®

Pixhawk: +16 years of community trust


















pixhawk®

Pixhawk: Latest Hardware



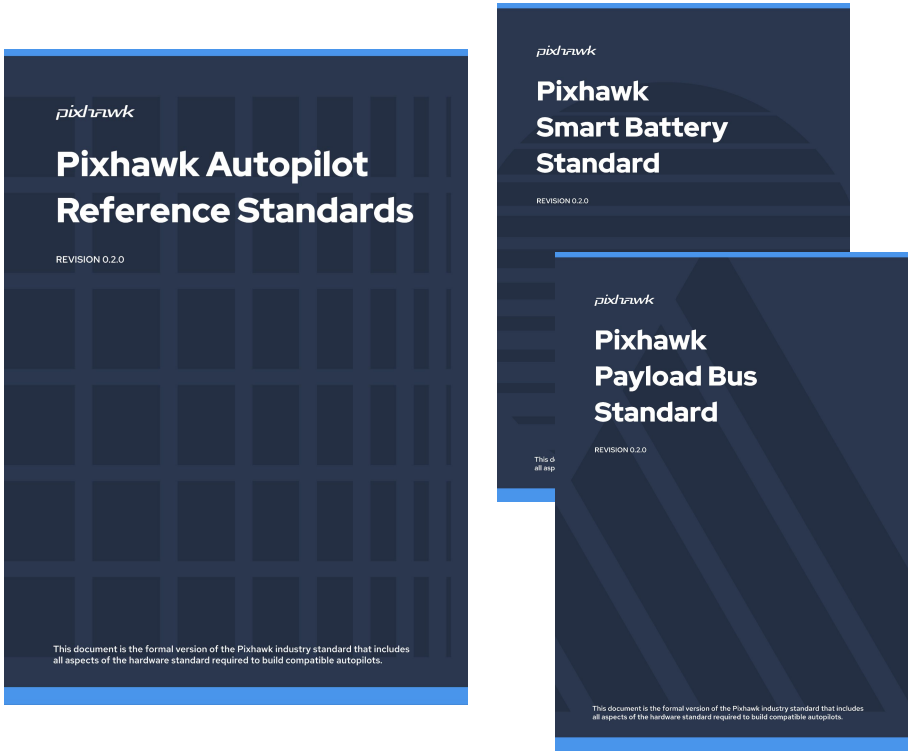
ARK Electronics ARKV6X
$400



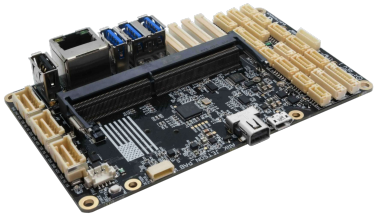
NXP MR-VMU-RT1176
$300

Pixhawk: Drone Open Standards

- The Pixhawk project evolved from an Open Hardware effort into an Open Standards project
- Our open standards are lifting the drone industry
- From serial connectors, to full blown flight controller architectures, we excel in quality and safety
- The latest generation of Pixhawk autopilots are the most advanced yet.
- https://pixhawk.org/

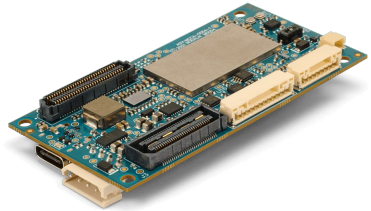


Pixhawk: AI-ready Compute Hardware



ARK Electronics
$1,100.00

ARK V6X
NVidia Jetson Family



ModalAI
$1,249.99

Qualcomm RB5



Holybro
$1,940

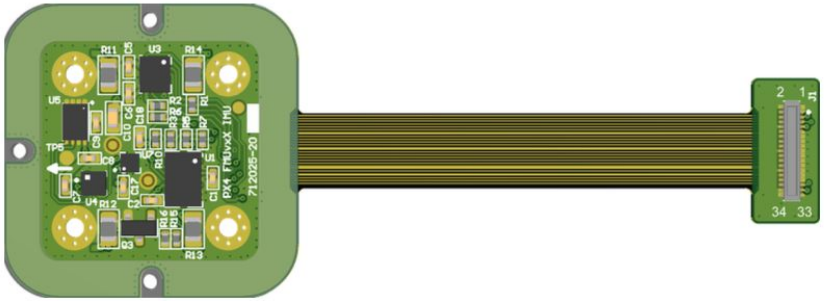
Pixhawk 6X
NVidia Jetson Family

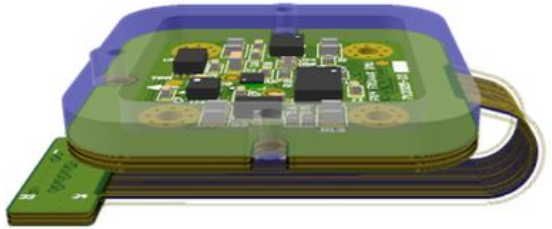


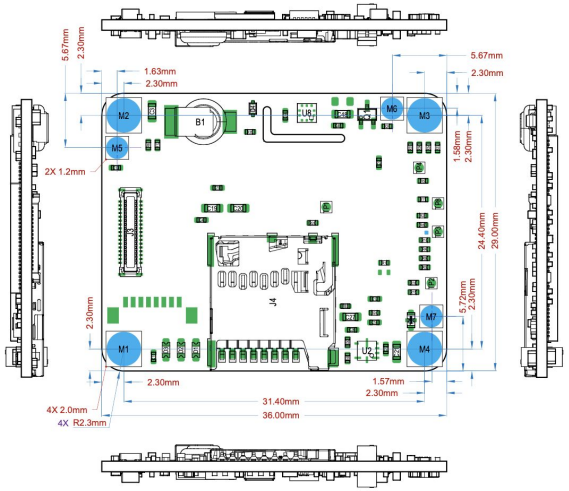
NXP
$299.00

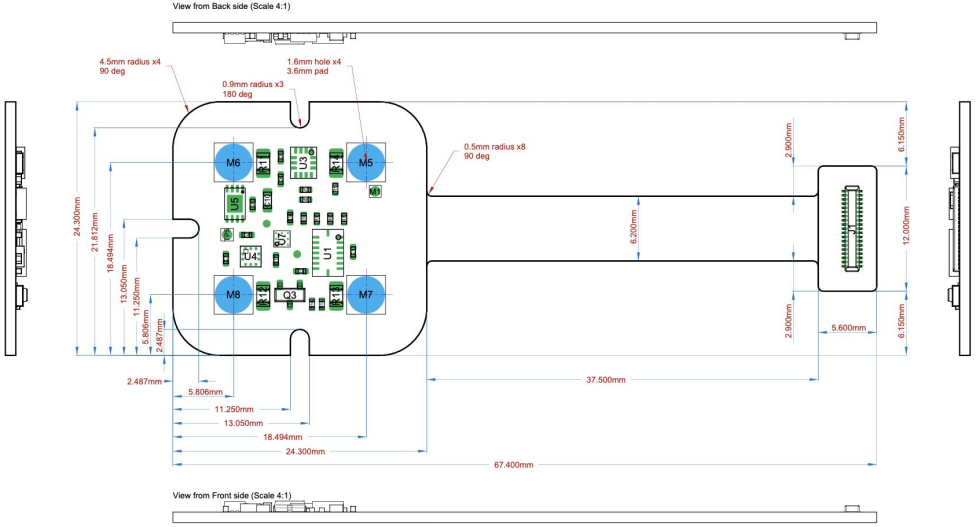
NavQPlus i.MX 8M Plus

Pixhawk Flight Controller FMUv6X

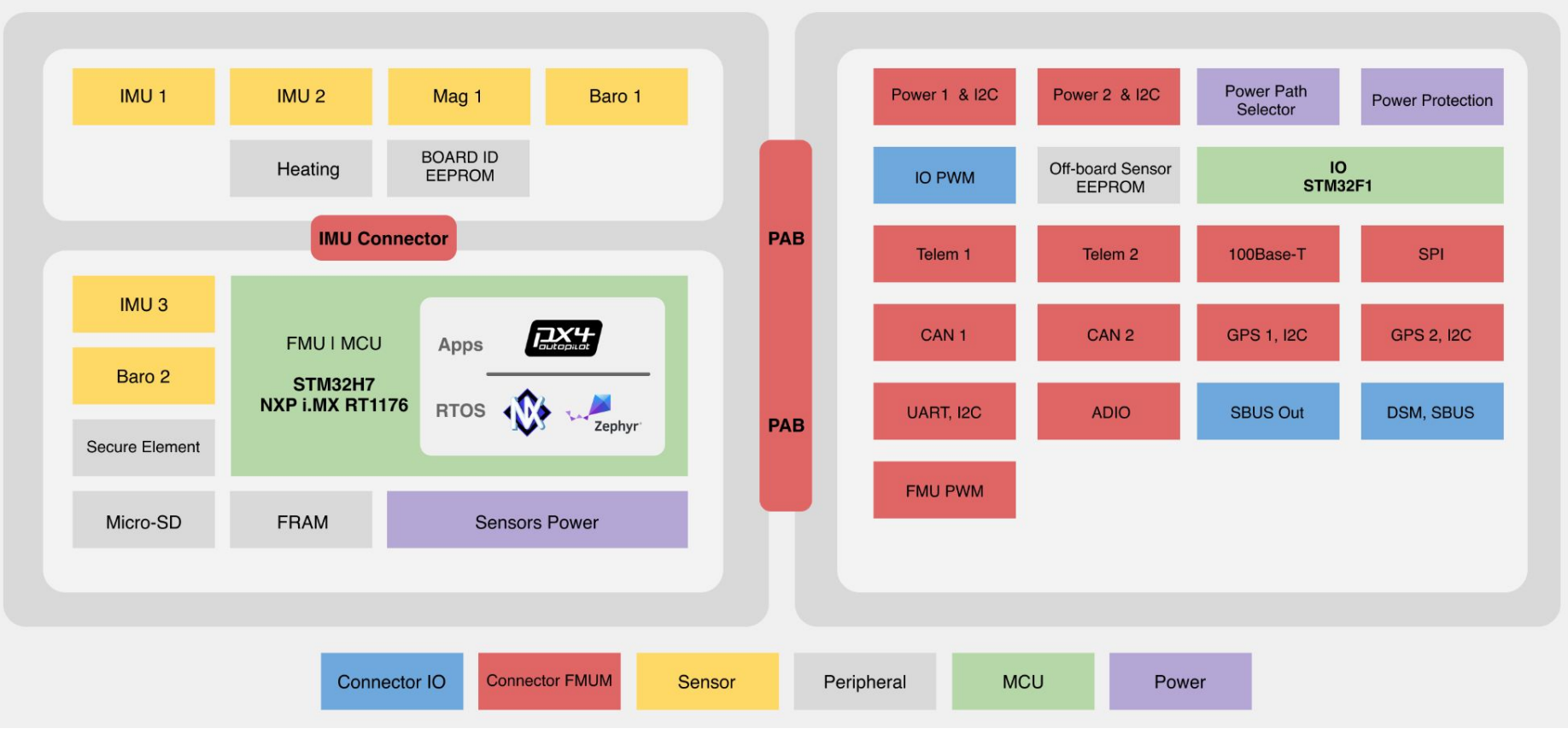








Pixhawk Flight Controller FMUv6X





MAVLINK

MICRO AIR VEHICLE COMMUNICATION PROTOCOL



MAVLink: The messaging protocol that connects it all

Open, lightweight messaging protocol

- One neutral steward
- Open governance, public dialect process, transparent releases
- Backwards compatibility taken seriously across a decade-plus
- Adopted across the industry, on every continent, by hundreds of vendors

- Publish Subscribe streams
- Point to Point sub protocols with retransmission
- Runs on every major autopilot and GCS
  - PX4 Autopilot, ArduPilot, QGroundControl, Mission Planner
- Microservices & Dialects

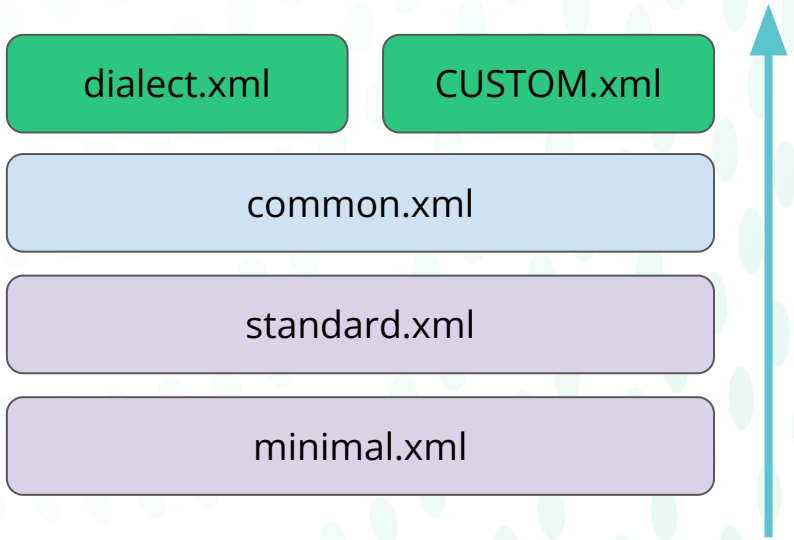
Seventeen years of backwards compatibility



MAVLink Dialects: extend without forking

Dialects are XML message set extensions layered on top of the core messages.

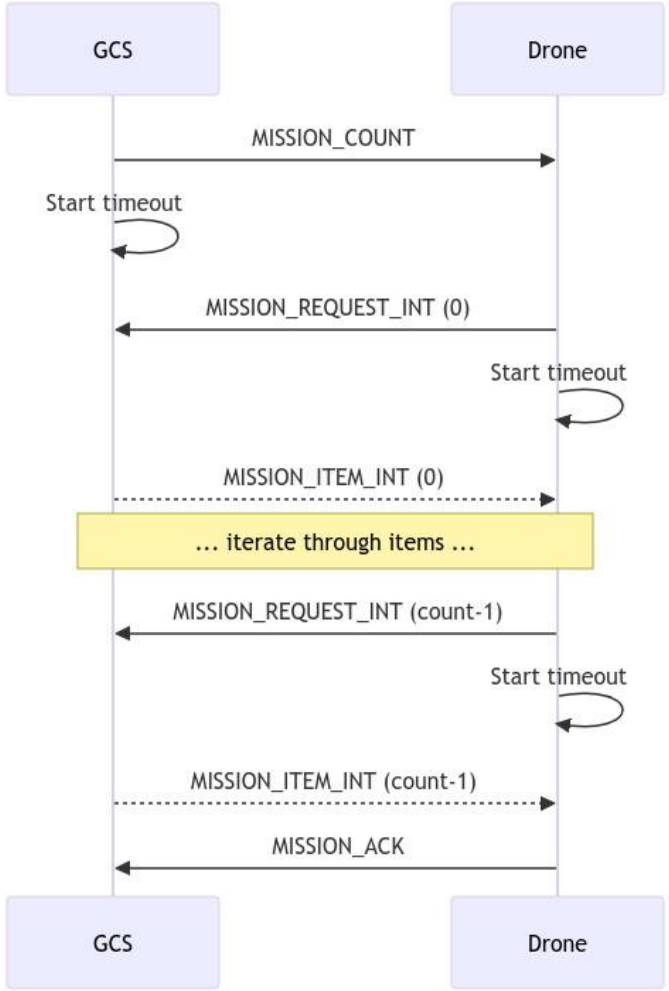
- A dialect includes common.xml and adds its own messages in an allocated ID range. No core message is touched, dialects never collide.
- Systems that don't speak a dialect ignore its messages and keep interoperating on the core set.
- Proven for over a decade: ArduPilot runs its own dialect (ardupilotmega.xml), vendors ship dialects for gimbals, ADS-B, and companion devices.
- Every improvement to the core is inherited automatically. Forks diverge. Dialects ride along.



MAVLINK
MICRO AIR VEHICLE COMMUNICATION PROTOCOL

MAVLink: Microservices

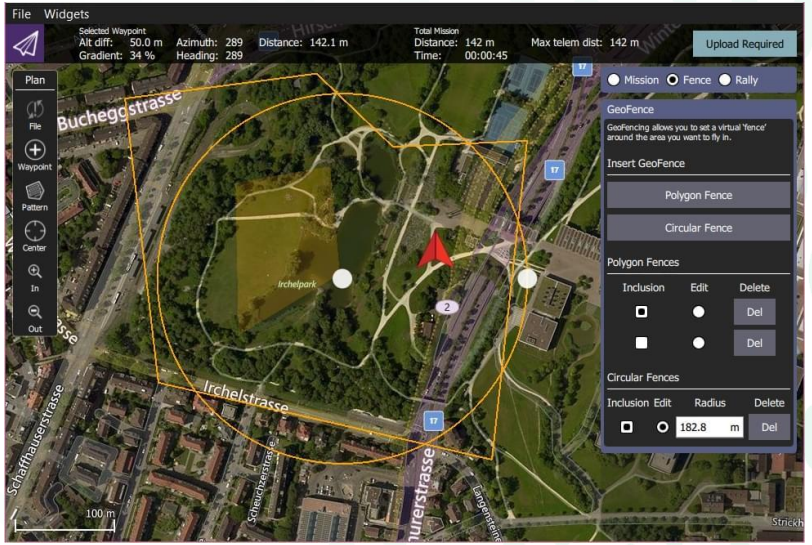
- Heartbeat/Connection
- Mission Protocol
- Parameter Protocol
- Camera Protocol
- Gimbal Protocol
- Battery Protocol
- Traffic Management
- Standard Flight Modes
- Time Synchronization
- Open Drone ID
- etc..



MAVLINK
MICRO AIR VEHICLE COMMUNICATION PROTOCOL

MAVLink: Ground Stations

```
<entry value="21" name="MAV_CMD_NAV_LAND" hasLocation="true" isDestination="true">
  <description>Land at location.</description>
  <param index="1" label="Abort Alt" units="m">Minimum target altitude if landing is aborted (0 = undefined/use system default).</param>
  <param index="2" label="Land Mode" enum="PRECISION_LAND_MODE">Precision land mode.</param>
  <param index="3">Empty.</param>
  <param index="4" label="Yaw Angle" units="deg">Desired yaw angle. NaN to use the current system yaw heading mode (e.g. yaw towards next way
  <param index="5" label="Latitude">Latitude.</param>
  <param index="6" label="Longitude">Longitude.</param>
  <param index="7" label="Altitude" units="m">Landing altitude (ground level in current frame).</param>
</entry>
<entry value="22" name="MAV_CMD_NAV_TAKEOFF" hasLocation="true" isDestination="true">
  <description>Takeoff from ground / hand. Vehicles that support multiple takeoff modes (e.g. VTOL quadplane) should take off using the curre
  <param index="1" label="Pitch" units="deg">Minimum pitch (if airspeed sensor present), desired pitch without sensor</param>
  <param index="2">Empty</param>
  <param index="3">Empty</param>
  <param index="4" label="Yaw" units="deg">Yaw angle (if magnetometer present), ignored without magnetometer. NaN to use the current system y
  <param index="5" label="Latitude">Latitude</param>
  <param index="6" label="Longitude">Longitude</param>
  <param index="7" label="Altitude" units="m">Altitude</param>
</entry>
<entry value="23" name="MAV_CMD_NAV_LAND_LOCAL" hasLocation="true" isDestination="true">
  <description>Land at local position (local frame only)</description>
  <param index="1" label="Target" minValue="0" increment="1">Landing target number (if available)</param>
  <param index="2" label="Offset" units="m" minValue="0">Maximum accepted offset from desired landing position - computed magnitude from sphe
  <param index="3" label="Descend Rate" units="m/s">Landing descend rate</param>
  <param index="4" label="Yaw" units="rad">Desired yaw angle</param>
  <param index="5" label="Y Position" units="m">Y-axis position</param>
  <param index="6" label="X Position" units="m">X-axis position</param>
  <param index="7" label="Z Position" units="m">Z-axis / ground level position</param>
</entry>
<entry value="24" name="MAV_CMD_NAV_TAKEOFF_LOCAL" hasLocation="true" isDestination="true">
  <description>Takeoff from local position (local frame only)</description>
  <param index="1" label="Pitch" units="rad">Minimum pitch (if airspeed sensor present), desired pitch without sensor</param>
  <param index="2">Empty</param>
  <param index="3" label="Ascend Rate" units="m/s">Takeoff ascend rate</param>
  <param index="4" label="Yaw" units="rad">Yaw angle (if magnetometer or another yaw estimation source present), ignored without one of these
  <param index="5" label="Y Position" units="m">Y-axis position</param>
  <param index="6" label="X Position" units="m">X-axis position</param>
  <param index="7" label="Z Position" units="m">Z-axis position</param>
</entry>
```



MAVLINK
MICRO AIR VEHICLE COMMUNICATION PROTOCOL



MAVSDK





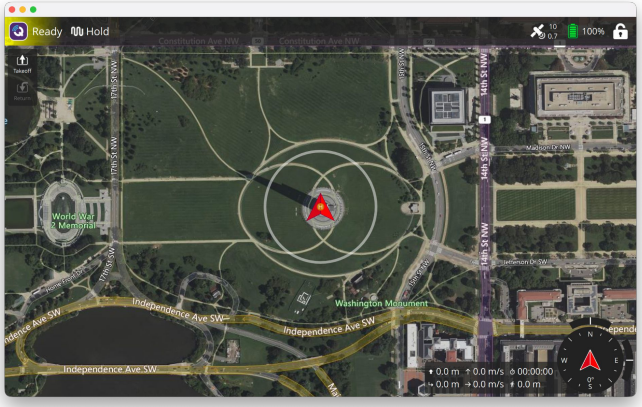
MAVSDK

- SDK for MAVLink
- Because you writing XML is not meant for humans.
- Available for the most widely used programming languages:
  - C++, Swift, Python, Java, Go, JS, CSharp, Rust
- Server / Client architecture
  - Both can be embedded in the same library
  - Server / Client communication is through gRPC
- Widely used and adopted by industry

Top Features I like

- Plugin Architecture
- Events Interface
- Error Handling
- Shared Vehicle Control
- Multiple resources available: example code and tutorials

GroundControl







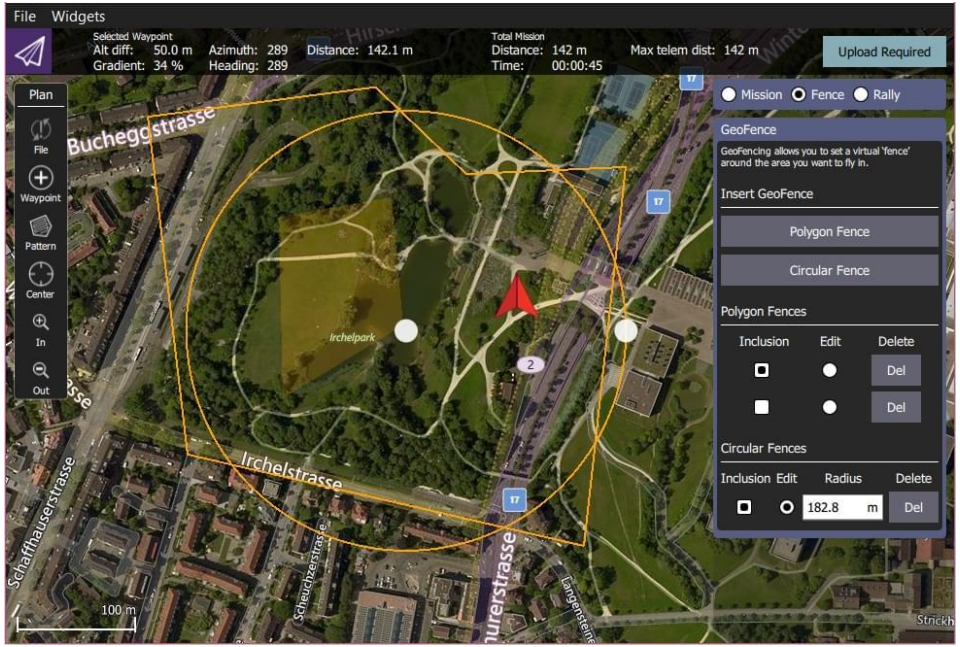
QGroundControl

Ground Station that features a wide range of functionality from all aspects of vehicle control for setup and tuning, to mission based control. All of this thanks to MAVLink

Can be deployed to multiple platforms: Linux, Windows, macOS, Android and iOS

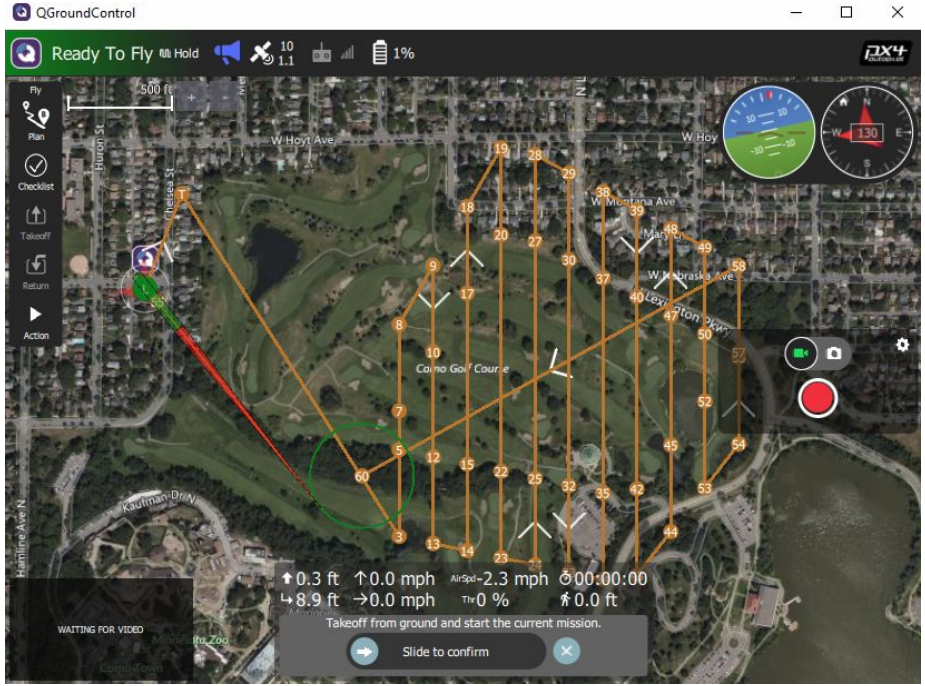
Vendors usually customize QGC for their end-users, allowing them to provide a fully featured Mission Control app.

Can be themed, you can remove any unwanted functionality.



QGroundControl

- **Open Source** Mission Planner
- Fully **themable** interface
- Support for all **MAVLink compliant** flight stacks
- **Vendor Neutral** governance
- Permissively licensed: **Apache 2.0**
- **Qt-based** application
- **Cross-platform**: Mobile, Tablets, Linux, Windows, and macOS
- Can control every aspect of a drone, **from firmware setup, to advanced mission planning**



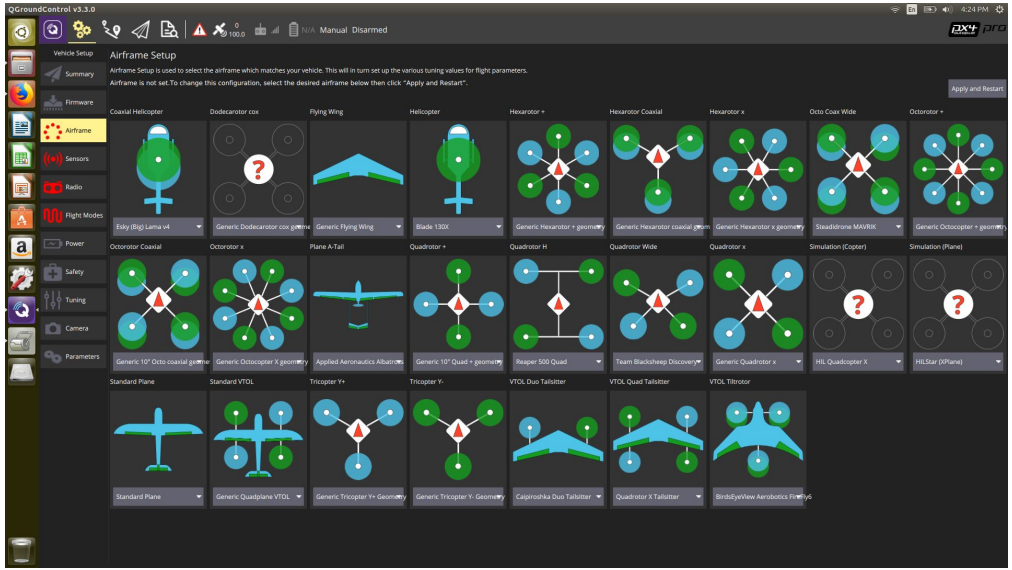

QGroundControl v. 0.9.0 (Alpha RC1) (vpn-global-
Unmanned Systems
MAV 001
AUTO MODE
WAYPOINT
ACTIVE
WP0
Active, normal operation.
Horizontal Situation Indicator
Flight Display
YAWSPEED D/S
AIRSPEED
CLIMBRATE
000.0
010.0
-004.0
Lat/Lon
Home
Edit
Set Home
Trails
Follow
Map View
Clear Trail
Reset
Sky
Day/Nigl
Mission Plan
NAV: Waypoint
Abs. Alt/Globa
lat 47.3780060°
lon 8.5470686°
alt 450.71m
lat 47.3778877°
lon 8.5481501°
alt 456.56m
lat 47.3773270°
lon 8.5498590°
alt 458.55m
lat 47.3768387°
lon 8.5485620°
alt 454.37m
lat 47.3763924°
lon 8.5488443°
alt 453.50m
Save WPs
Load WPs
Read
Write
New current waypoint 0
Imagery Date: 7/1/2009
Eye alt 621 m
84 m
Image © 2011 GeoContent
Terms of Use


SAMSUNG
Manual
Disarmed
3.5
2359.1

QGroundControl

Top Features I like

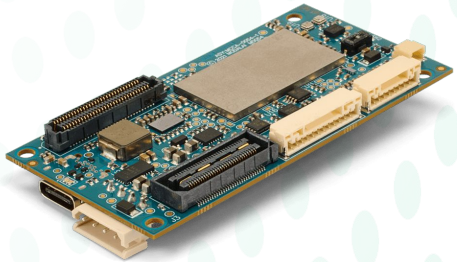
- Full setup and control of PX4 vehicles. (Also supports other MAVLink based platforms)
- Mission Planning for Autonomous Flight
- Flight Map with all telemetry, waypoints and vehicle instruments.
- 3D Viewer
- Video Streaming
- Support for managing multiple Vehicles
- Multi-Platform



PX4 autopilot

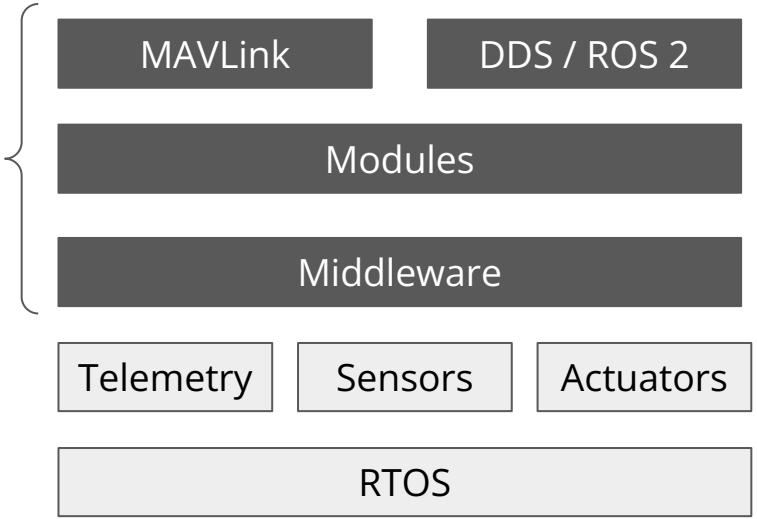






PX4 Autopilot

- **Open Source** Autopilot Stack
- Developer friendly with **vibrant community** participation
- Support for a wide range of **boards** and **sensors**
- **Vendor Neutral** governance
- Business friendly license: **BSD-3**
- **Modular** architecture with a **DDS-compatible middleware** (uORB)
- Modules are fully **parallelized**, and **thread safe**
- Support for custom builds, **trim what you don't need**



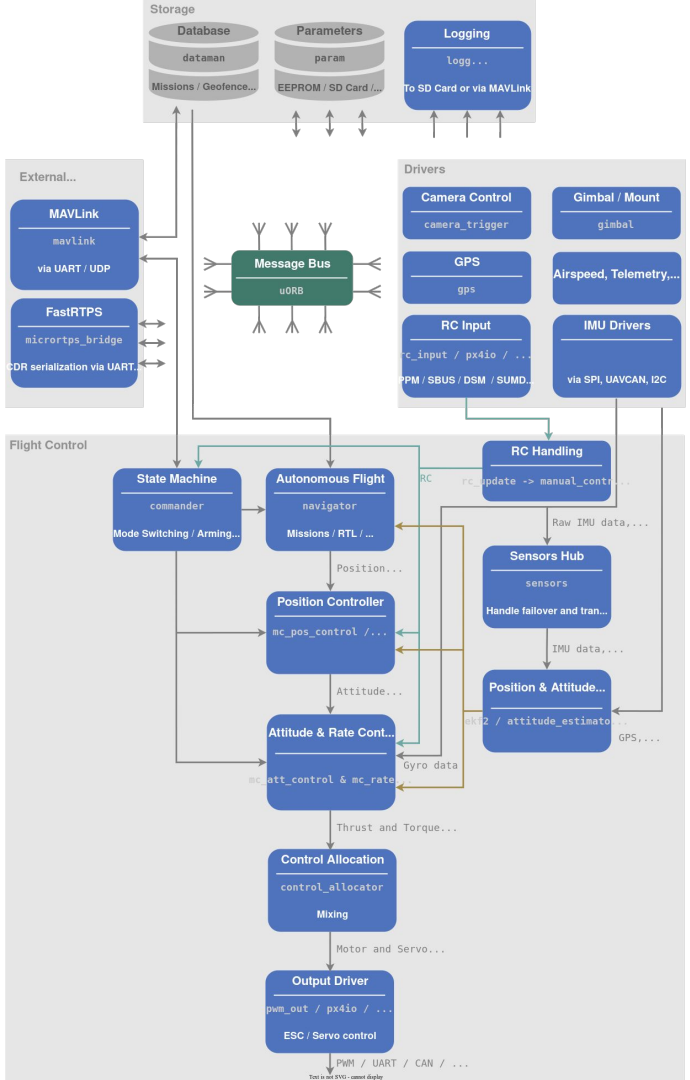
PX4 Autopilot

- **Multi-Vehicle** type support
  - **Multicopter**
  - **Fixed Wing**
  - **VTOL**
  - **Rover**
  - **Under Surface**
  - **Above Surface**
  - **Balloons**
  - **Satellites**
  - **Jetpack**

- **Developer Friendly Interfaces**
  - **Flight Modes** provide a set of helpers to control autonomy
  - **Flight Tasks** allowing developers to extend flight modes
  - **Parameter** database exposing functionality back to users
  - **Events interface** giving developers a system-wide API for notifications
- **Control allocation** translates thrust and torque commands into actuator commands which control motors and servos
  - Controllers do not require special handling for airframe geometry
- Native **ROS 2 Support** through DDS

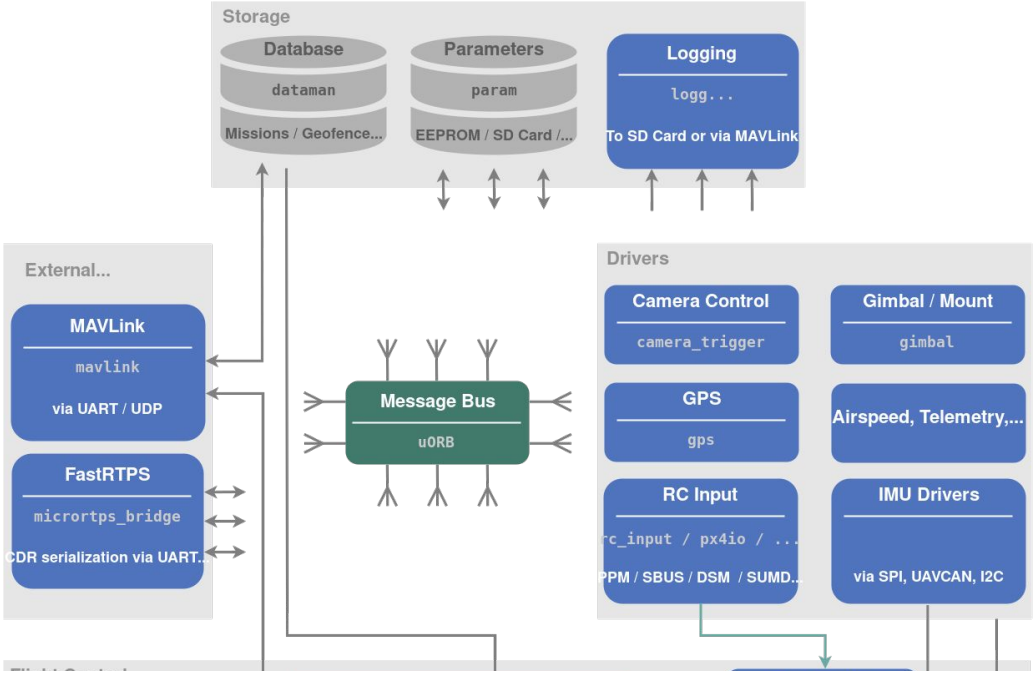
PX4's Stack Architecture

- **Developer Friendly Interfaces**
  - **Flight Modes** provide a set of helpers to control autonomy
  - **Flight Tasks** allowing developers to extend flight modes
  - **Parameter** database exposing functionality back to users
  - **Events interface** giving developers a system-wide API for notifications
- **Control allocation** translates thrust and torque commands into actuator commands which control motors and servos
  - Controllers do not require special handling for airframe geometry
- Native **ROS 2 Support** through DDS



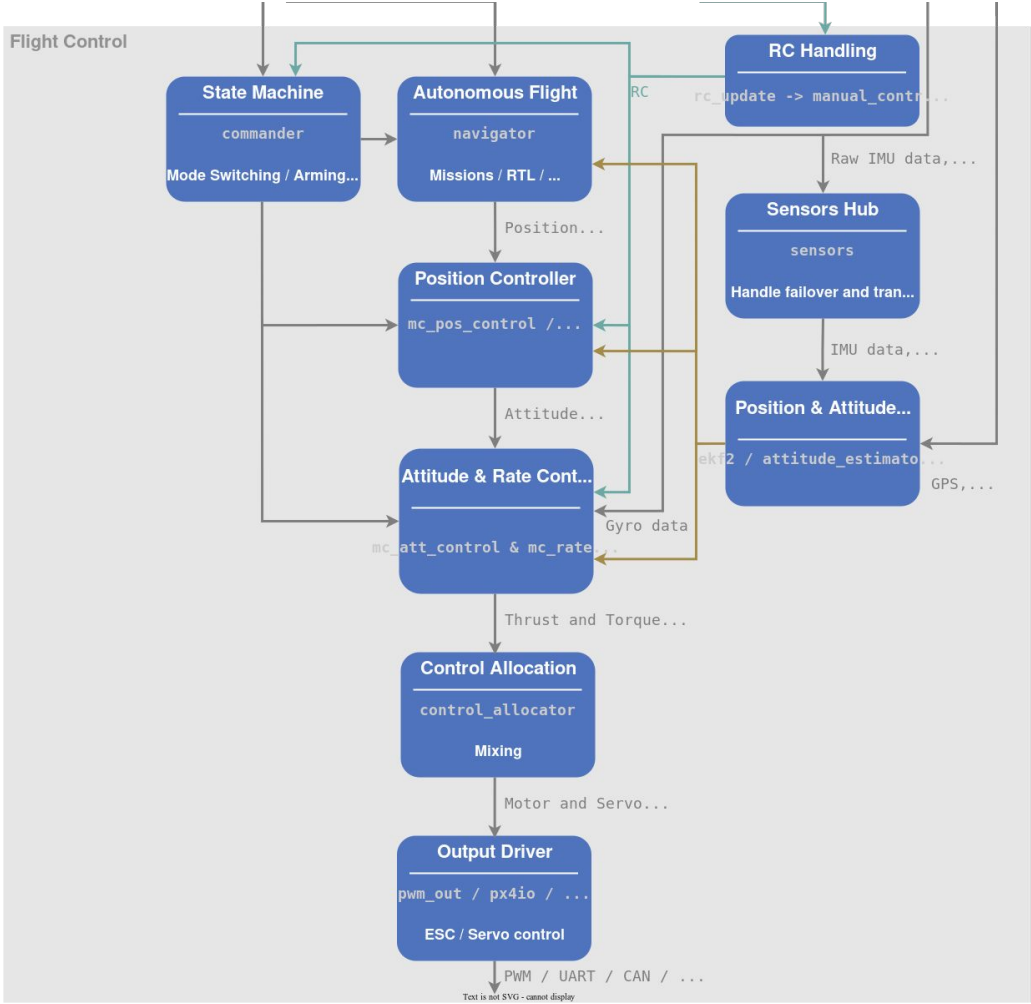
PX4's Stack Architecture

- System Storage:
  - Database
  - Parameters
  - Logging
- Drivers:
  - GPS
  - Airspeed
  - IMU
  - Baro
  - etc...
- External Interfaces:
  - MAVLink
  - XRCE-DDS for ROS 2



PX4's Stack Architecture

- State Machine
- Sensor Hub
- Autonomous Flight (Navigation)
  - Position Controller
  - Attitude & Rate Controller
  - Control Allocation
- RC Handling



Parameter based configuration

The burden of such a generic platform falls upon our Parameter system. Parameters are stored in the filesystem. We have two types of parameters: Runtime and Boot time.

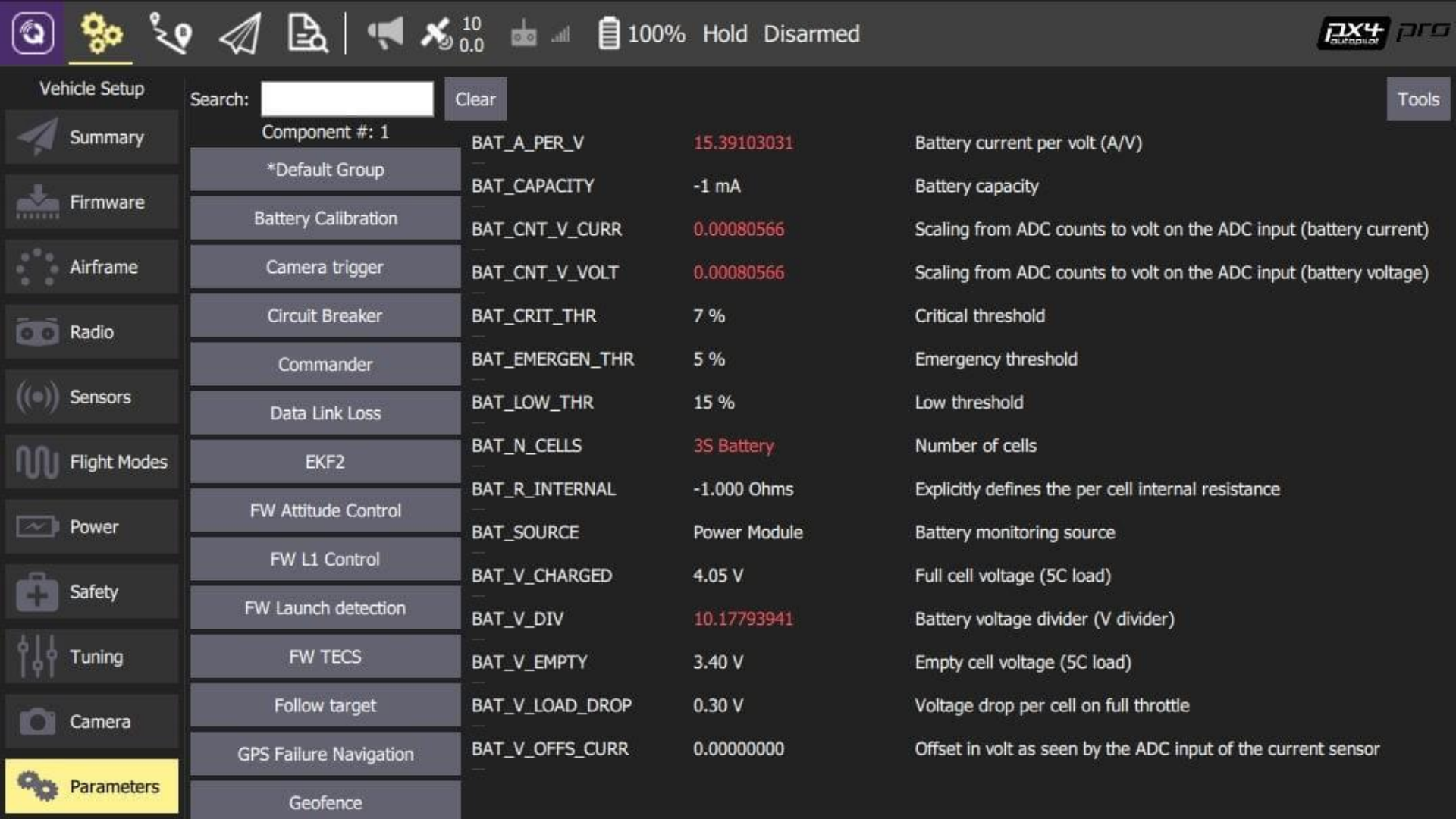
Main Concepts

- Data types: **float** & **int32_t**
- You can export / import parameters
- YAML metadata & C/C++ API

Everything can be configurable approach

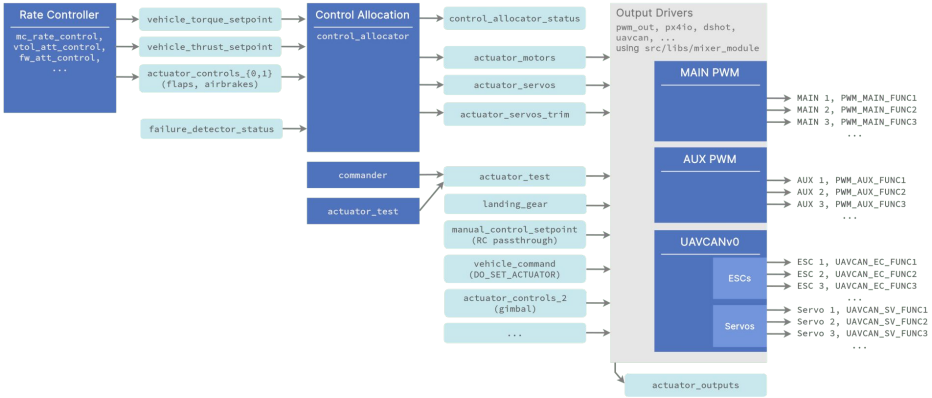
What works for us?

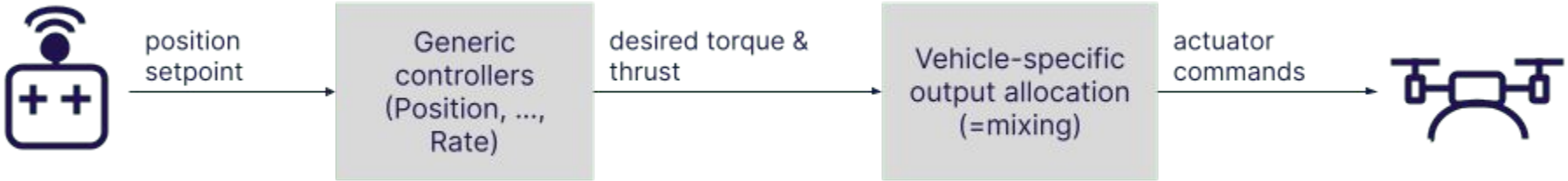
- We give developers the tools to allow their end users to configure aspects of their system
- Think of how to a pilot would define the takeoff altitude
- Really powerful when paired with a ground station
- Thanks to metadata we can build cool online reference lists


10
0.0
100% Hold Disarmed
Vehicle Setup
Summary
Firmware
Airframe
Radio
Sensors
Flight Modes
Power
Safety
Tuning
Camera
Parameters
Search:
Clear
Tools
Component #: 1
*Default Group
Battery Calibration
Camera trigger
Circuit Breaker
Commander
Data Link Loss
EKF2
FW Attitude Control
FW L1 Control
FW Launch detection
FW TECS
Follow target
GPS Failure Navigation
Geofence
BAT_A_PER_V 15.39103031 Battery current per volt (A/V)
BAT_CAPACITY -1 mA Battery capacity
BAT_CNT_V_CURR 0.00080566 Scaling from ADC counts to volt on the ADC input (battery current)
BAT_CNT_V_VOLT 0.00080566 Scaling from ADC counts to volt on the ADC input (battery voltage)
BAT_CRIT_THR 7 % Critical threshold
BAT_EMERGEN_THR 5 % Emergency threshold
BAT_LOW_THR 15 % Low threshold
BAT_N_CELLS 3S Battery Number of cells
BAT_R_INTERNAL -1.000 Ohms Explicitly defines the per cell internal resistance
BAT_SOURCE Power Module Battery monitoring source
BAT_V_CHARGED 4.05 V Full cell voltage (5C load)
BAT_V_DIV 10.17793941 Battery voltage divider (V divider)
BAT_V_EMPTY 3.40 V Empty cell voltage (5C load)
BAT_V_LOAD_DROP 0.30 V Voltage drop per cell on full throttle
BAT_V_OFFS_CURR 0.00000000 Offset in volt as seen by the ADC input of the current sensor

Control Allocation (mixing)

Control allocation in PX4 maps desired forces and torques from the controller into actuator outputs like motors or servos. It abstracts the vehicle geometry, so the same control logic works for multirotors, planes, or hybrids by handling the mixing and actuator mapping behind the scenes.





Geometry: Multirotor **Basic**

Motors 4

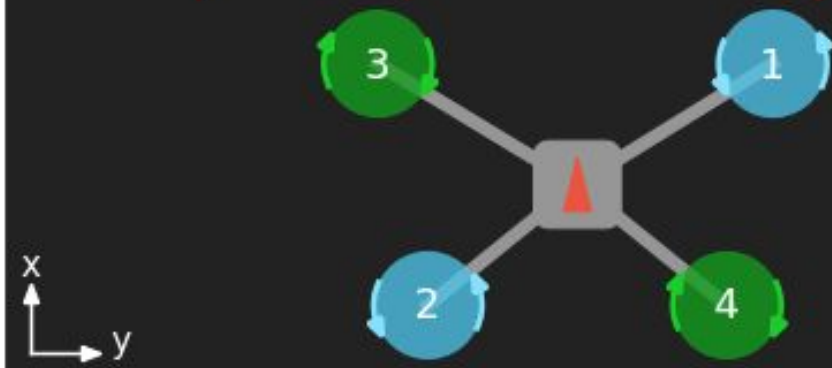
	Position X	Position Y	Direction CCW
Motor 1:	0.15	0.25	☑
Motor 2:	-0.15	-0.19	☑
Motor 3:	0.15	-0.25	☐
Motor 4:	-0.15	0.19	☐



Geometry: Multirotor **Advanced**

Motors 4

	Bidirectional	Slew Rate	Position X	Position Y	Position Z	Direction CCW
Motor 1:	☐	0.00	0.15	0.25	0.00	☑
Motor 2:	☐	0.00	-0.15	-0.19	0.00	☑
Motor 3:	☐	0.00	0.15	-0.25	0.00	☐
Motor 4:	☐	0.00	-0.15	0.19	0.00	☐

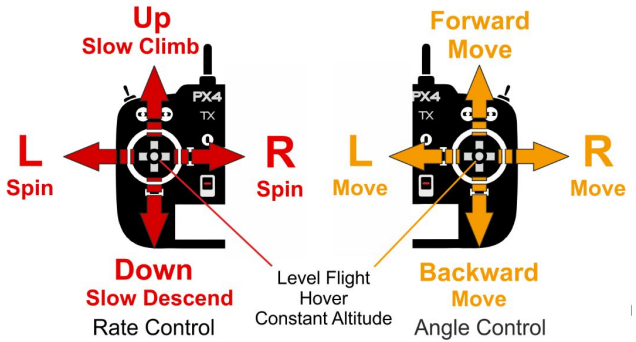


PX4's *Internal* Flight Modes

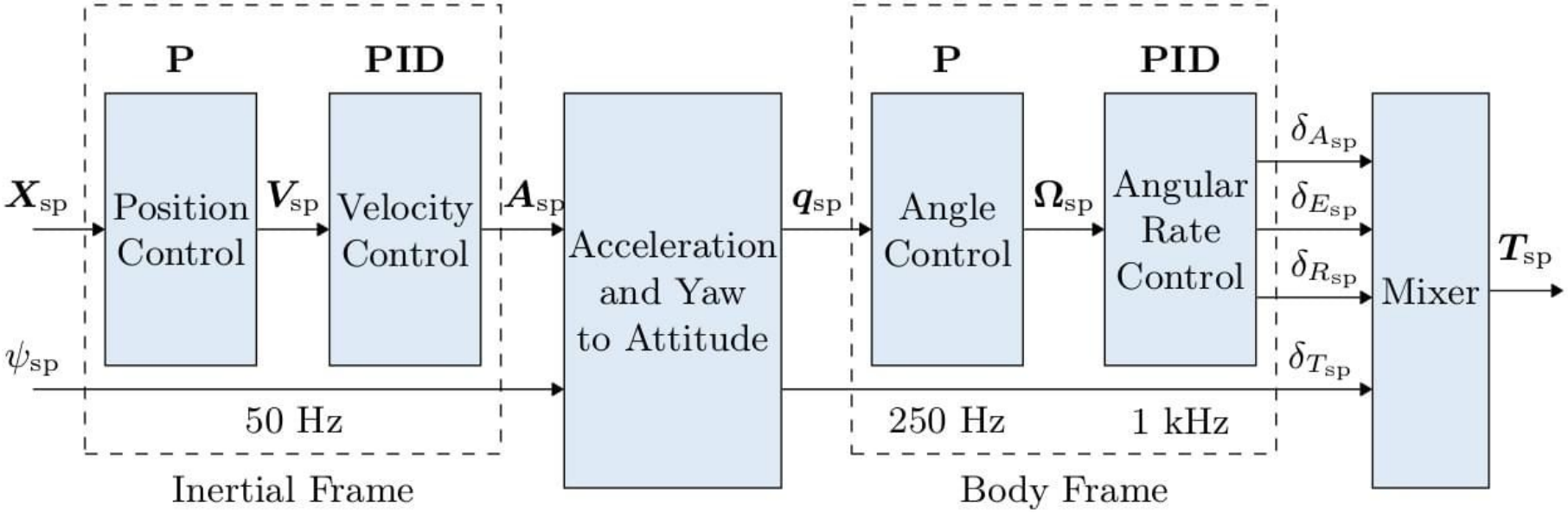
Running realtime inside the Embedded Controller.

- **Position**: When the sticks are released the vehicle will remain locked to a position in 3D space.
- **Altitude**: When the sticks are released the vehicle will maintain current altitude.
- **Hold**: Vehicle stops and hovers at its current GPS position.
- **Takeoff**: Vehicle takes off to a specified height and waits further input.
- **Land**: Lands vehicle at the position where the mode was engaged.
- **Mission**: Starts a mission that was preloaded to the drone

- Modes can be Manual or Autonomous
- The API to create modes is "**Flight Task**"



Controller diagrams, Multicopters



uORB Middleware - Connecting apps!

u(Micro) Object Request Broker

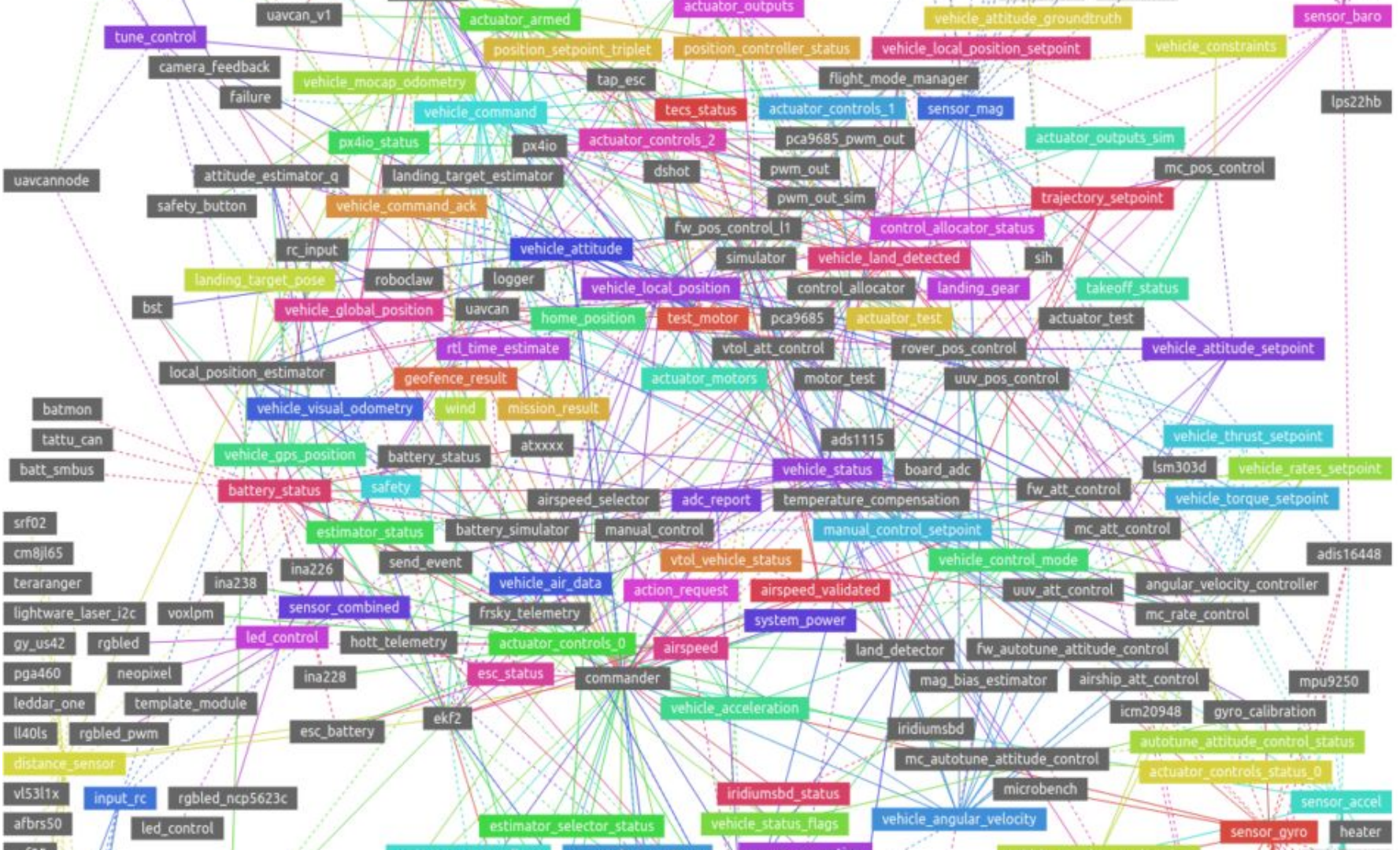
It's a really simple middleware, and arguably the top feature of PX4, at least in my opinion. It's one of the first applications that launch on boot since most of PX4 depends on it.

Main Concepts

- Messages - field information, typically reused by multiple topics
- Topics - what you subscribe and publish
- Modules - apps that depend on pub/sub of topics

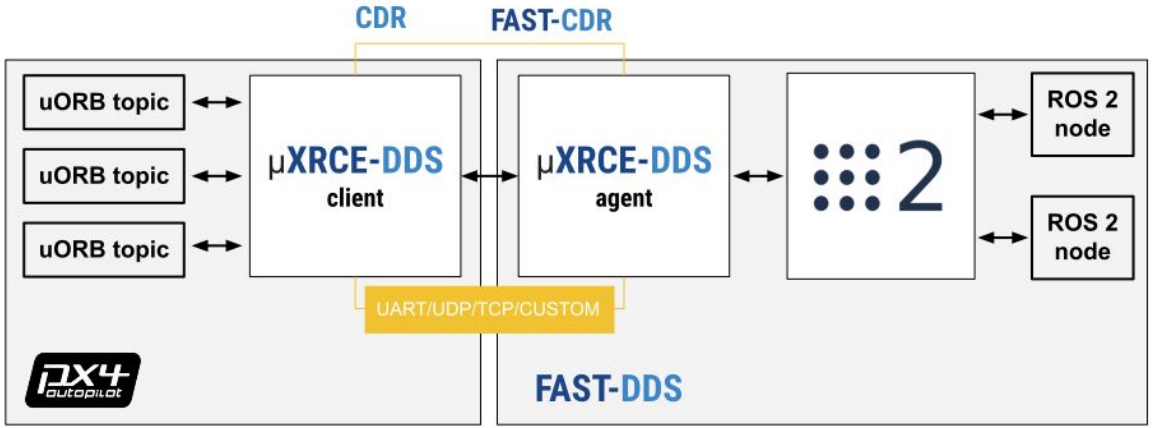
What we like about uORB?

- Fast!
- DDS-like... compatible
- Offers a comprehensive way to build interfaces between modules/apps
- Priorities, rates, queues
- Messages are defined in **.msg** and generated at build time (**.h** / **.cpp**)
- NuttX Shell (NSH) Tooling
  - `ls /obj`
  - `listener <message_name> <num_messages>`
  - `uorb top`


uavcan_v1
actuator_armed
actuator_outputs
vehicle_attitude_groundtruth
sensor_baro
tune_control
position_setpoint_triplet
position_controller_status
vehicle_local_position_setpoint
vehicle_constraints
camera_feedback
vehicle_mocap_odometry
tap_esc
flight_mode_manager
failure
vehicle_command
tecs_status
actuator_controls_1
sensor_mag
lps22hb
px4io_status
px4io
actuator_controls_2
pca9685_pwm_out
actuator_outputs_sim
uavcannode
attitude_estimator_q
landing_target_estimator
dshot
pwm_out
mc_pos_control
safety_button
vehicle_command_ack
pwm_out_sim
trajectory_setpoint
fw_pos_control_l1
control_allocator_status
rc_input
vehicle_attitude
simulator
vehicle_land_detected
sih
landing_target_pose
roboclaw
logger
vehicle_local_position
control_allocator
landing_gear
takeoff_status
bst
vehicle_global_position
uavcan
home_position
test_motor
pca9685
actuator_test
actuator_test
rtl_time_estimate
vtol_att_control
rover_pos_control
vehicle_attitude_setpoint
local_position_estimator
geofence_result
actuator_motors
motor_test
uuv_pos_control
batmon
vehicle_visual_odometry
wind
mission_result
tattu_can
ads1115
vehicle_thrust_setpoint
vehicle_gps_position
battery_status
atxxxx
batt_smbus
vehicle_status
board_adc
lsm303d
vehicle_rates_setpoint
battery_status
safety
airspeed_selector
adc_report
temperature_compensation
fw_att_control
vehicle_torque_setpoint
srf02
estimator_status
battery_simulator
manual_control
manual_control_setpoint
mc_att_control
cm8jl65
send_event
vtol_vehicle_status
vehicle_control_mode
adis16448
teraranger
ina226
ina238
vehicle_air_data
action_request
airspeed_validated
uuv_att_control
angular_velocity_controller
lightware_laser_i2c
voxlpm
sensor_combined
frsky_telemetry
system_power
mc_rate_control
gy_us42
rgbled
led_control
hott_telemetry
actuator_controls_0
airspeed
land_detector
fw_autotune_attitude_control
pga460
neopixel
ina228
esc_status
commander
mag_bias_estimator
airship_att_control
mpu9250
leddar_one
template_module
ekf2
vehicle_acceleration
icm20948
gyro_calibration
ll40ls
rgbled_pwm
esc_battery
iridiumsbd
autotune_attitude_control_status
distance_sensor
mc_autotune_attitude_control
actuator_controls_status_0
vl53l1x
input_rc
rgbled_ncp5623c
iridiumsbd_status
microbench
sensor_accel
afbrs50
led_control
estimator_selector_status
vehicle_status_flags
vehicle_angular_velocity
sensor_gyro
heater

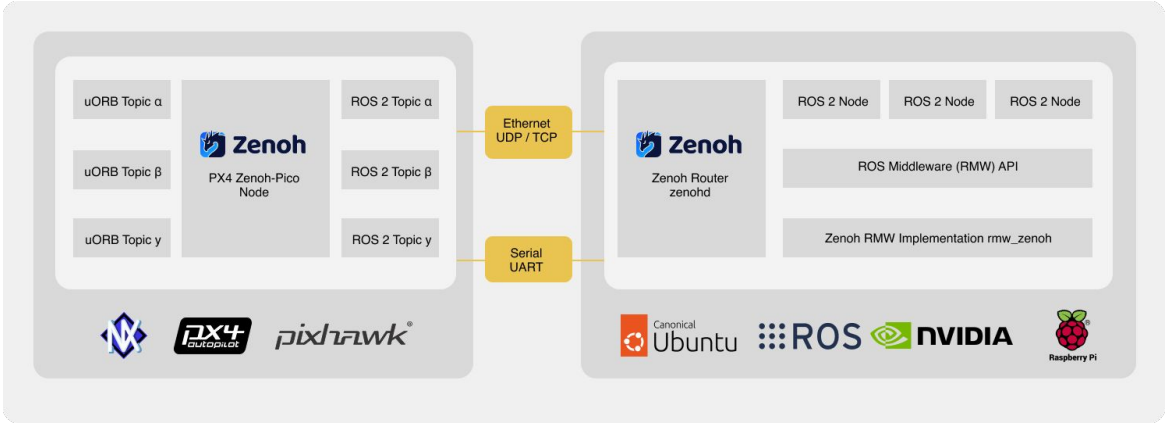
ROS 2 Support - DDS

- Thanks to uORB middleware we can communicate directly with the ROS 2 middleware (XRCE-DDS based)
- PX4 internal modules can share data with ROS 2 nodes
- Agent / Client approach
- Ethernet and Serial support
- ROS 2 QoS Supported!



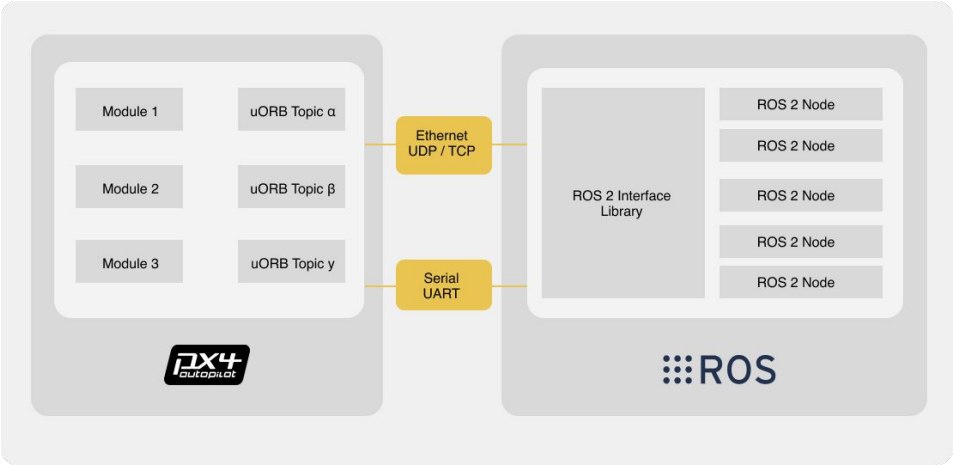
ROS 2 Support - Zenoh

- Direct uORB ↔ ROS 2 data exchange
- Uses Zenoh-pico client on PX4
- Connects via TCP, UDP, or UART (multicast supported)
- Works with rmw_zenoh
- Topic mapping via SD card config
- Experimental but ready for testing



ROS 2 Interface Library

- C++ library bridging ROS 2 with PX4
- Three high-level interfaces:
  - Control → send set-points / actuate vehicle
  - Navigation → feed pose / odometry
  - Waypoint Missions → run full mission logic
- Enables ROS 2-native modes that register with PX4 and look like built-in modes.
- DDS ready
- Zenoh experimental



External Flight Modes

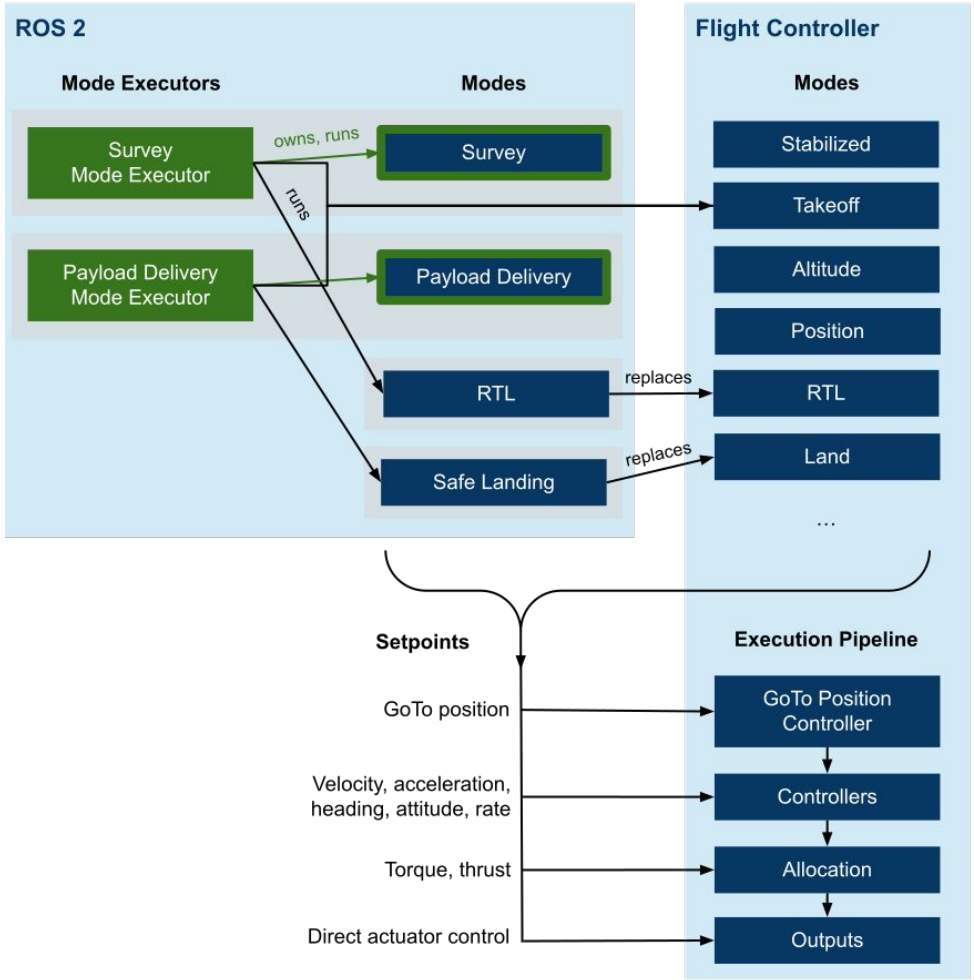
External Flight Modes are Flight Modes that can be developed and executed from outside the Flight Controller and in a non-real time system.

When do External Flight Modes matter?

- When you don't need low-level access for direct motor control
- When you have a compute module where you can run ROS 2
- When the mode is not Safety Critical

Advantages

- You don't need to deal with low level embedded constraints and requirements
- Crashing the app won't result in a catastrophic failure
- Easier to maintain outside of the PX4 codebase
- Take advantage of high-level functionality like dynamic data structures, and the ROS 2 library ecosystem.
- More advanced compute for greater processing

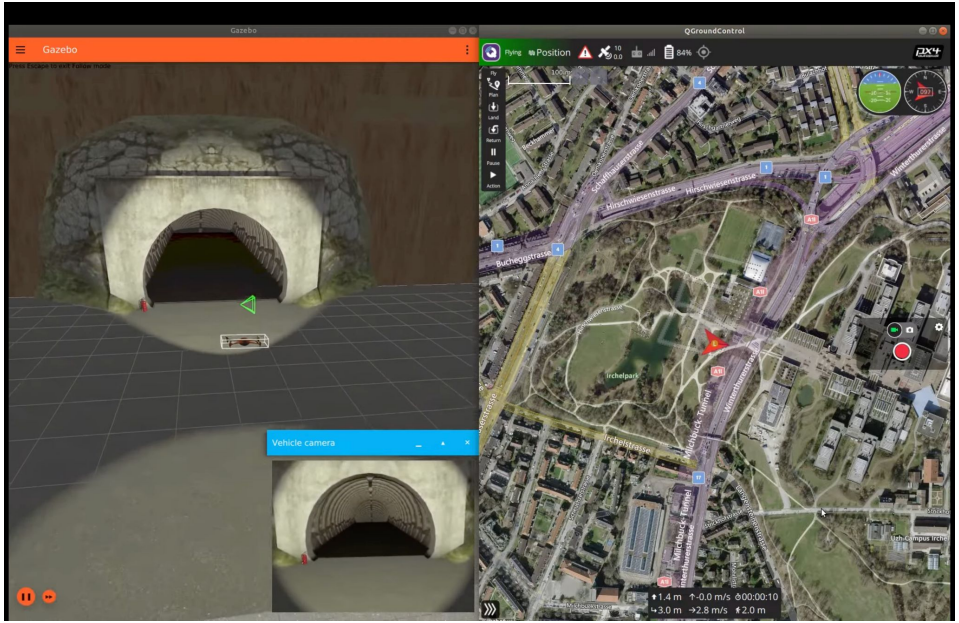


PX4 Simulation

Gazebo is our default simulation agent, we support both classic and modern gazebo, with multiple worlds and models to choose from.

Other Simulation Engines Supported

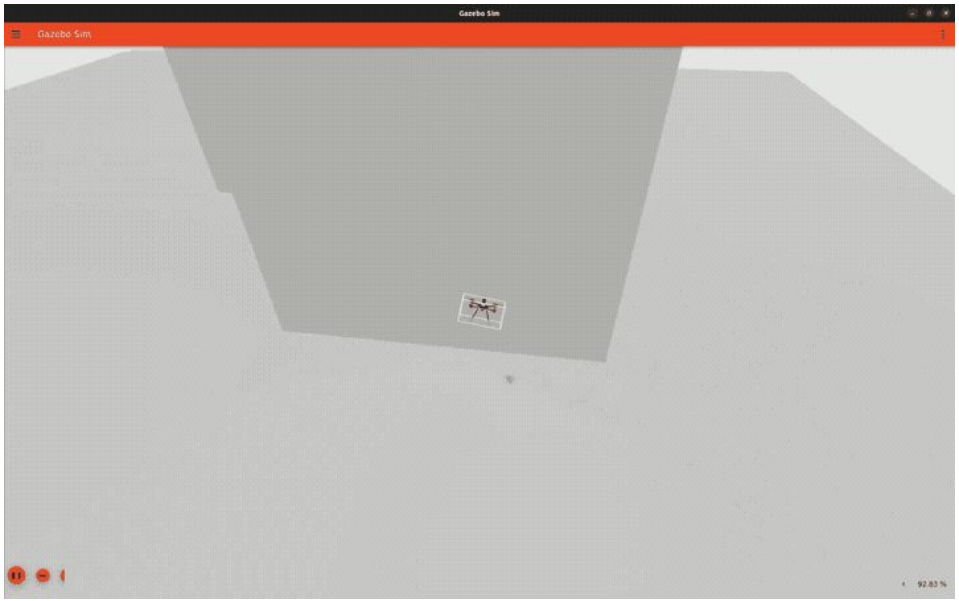
- Gazebo Classic
- AirSim
- Flight Gear
- jMAVSim



Gazebo Simulation Docs

PX4 Simulation

Gazebo is our default simulation agent, we support both classic and modern gazebo, with multiple worlds and models to choose from.



Other Simulation Engines Supported

- Gazebo Classic
- AirSim
- Flight Gear
- jMAVSim

Gazebo Simulation Docs

Supported Hardware Architectures



Cortex-M4, Cortex-M7 &
Cortex-M7 dual-core



32 & 64 bit

Supported RTOS



NuttX RTOS



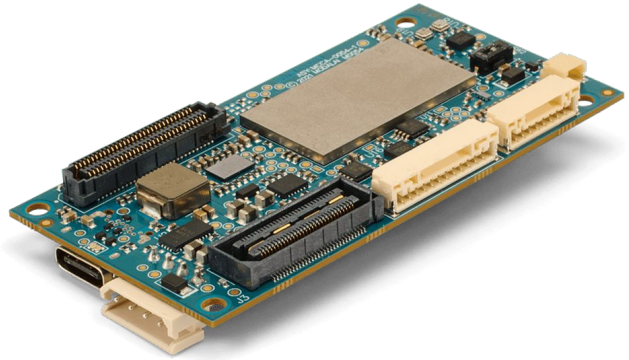
Zephyr®

(*Coming Soon)

Hardware Support

- Support for more than 80 boards from 30+ manufacturers
- Drivers for more than 100+ sensors
  - IMU, Baro, Actuators, GPS, INS, CAN, UWB... etc.
- Main Architectures Supported
  - STM32 - STMicro
  - Snapdragon - Qualcomm
  - iMX - NXP Semiconductors
  - RISCV-V





Official PX4 Developer Kits



DroneBlocks
$1,999.00



ModalAI
$2,749.99



Holybro
$507.99

Quick Break

Coming Next: Developer Environment

Aerial Robotics Meetup, tonight!

Tonight, 8:00 to 11:00 PM
CRAFT Beer Market, 1 Adelaide St E (short walk from the venue)
Sponsored by Dronecode. **Food and drinks are on us.**

Talk shop on control architectures, state estimation, and open source flight stacks with the people who actually maintain them.



Prerequisites - environment setup - architecture overview

In this section you will

- Understand the requirements for running the exercise
- Set up the simulation environment
- Fly a drone in Gazebo without ROS 2
- Attach ROS 2 to the simulation
- Echo PX4 topics in ROS 2
- Visualize PX4 data with Foxglove

Environment Setup

REQUIREMENTS:

- Install Foxglove `https://foxglove.dev/download`
- Download and install QGC `https://qgroundcontrol.com`
- Clone PX4 repo: https://docs.px4.io/main/en/sim_gazebo_gz/#installation-ubuntu-linux

```
git clone -b release/1.18 https://github.com/PX4/PX4-Autopilot.git
```

- Prepare workspace and clone workshop repo https://github.com/Dronecode/roscon-25-workshop :

```
mkdir -p ~/workspaces/px4_roscon26_ws/src
cd ~/workspaces/px4_roscon26_ws/src
git clone git@github.com:Dronecode/roscon-25-workshop.git px4-roscon-workshop
cd ~/workspaces/px4_roscon26_ws
vcs import src < ./src/px4-roscon-workshop/jazzy.repos
```

Environment Setup, dockerless, Ubuntu 22.04/24.04, WSL 2

- Follow PX4 simulation setup instructions https://docs.px4.io/main/en/sim_gazebo_gz/#installation-ubuntu-linux

  To setup PX4 SITL build toolchain and download Gazebo Harmonic

  Verify it by starting a simple simulation over Gazebo, no ROS 2 involved:

  ```
  cd ~/PX4-Autopilot
  make px4_sitl gz_x500
  ```

- Install ROS 2 (Jazzy or Humble depending on OS version)

  https://docs.ros.org/en/jazzy/Installation/Ubuntu-Install-Debs.html. Pick `ros-jazzy-desktop`

  https://docs.ros.org/en/humble/Installation/Ubuntu-Install-Debs.html. Pick `ros-humble-desktop`

- Install ROS 2 developer tools: `sudo apt install ros-dev-tools`
- Install dependencies:

  ```
  cd ~/workspaces/px4_roscon26_ws
  source /opt/ros/jazzy/setup.bash # source /opt/ros/hummble/setup.bash
  rosdep install -i --from-paths src --skip-keys OpenCV
  ```

Environment Setup, docker

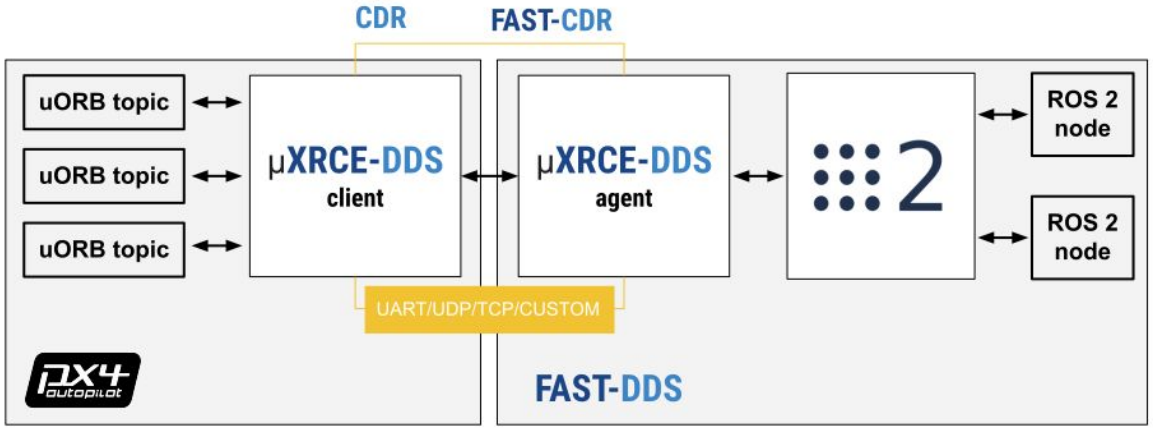
- Retrieve `px4io/px4-dev-ros2-gazebo:main-jazzy` docker image or build it yourself
  https://github.com/PX4/PX4-Autopilot/pull/28730
- Start the container

```
cd ~/workspaces/px4_roscon26_ws/src/ px4-roscon-workshop
./docker/docker_run.sh ~/PX4-Autopilot
```

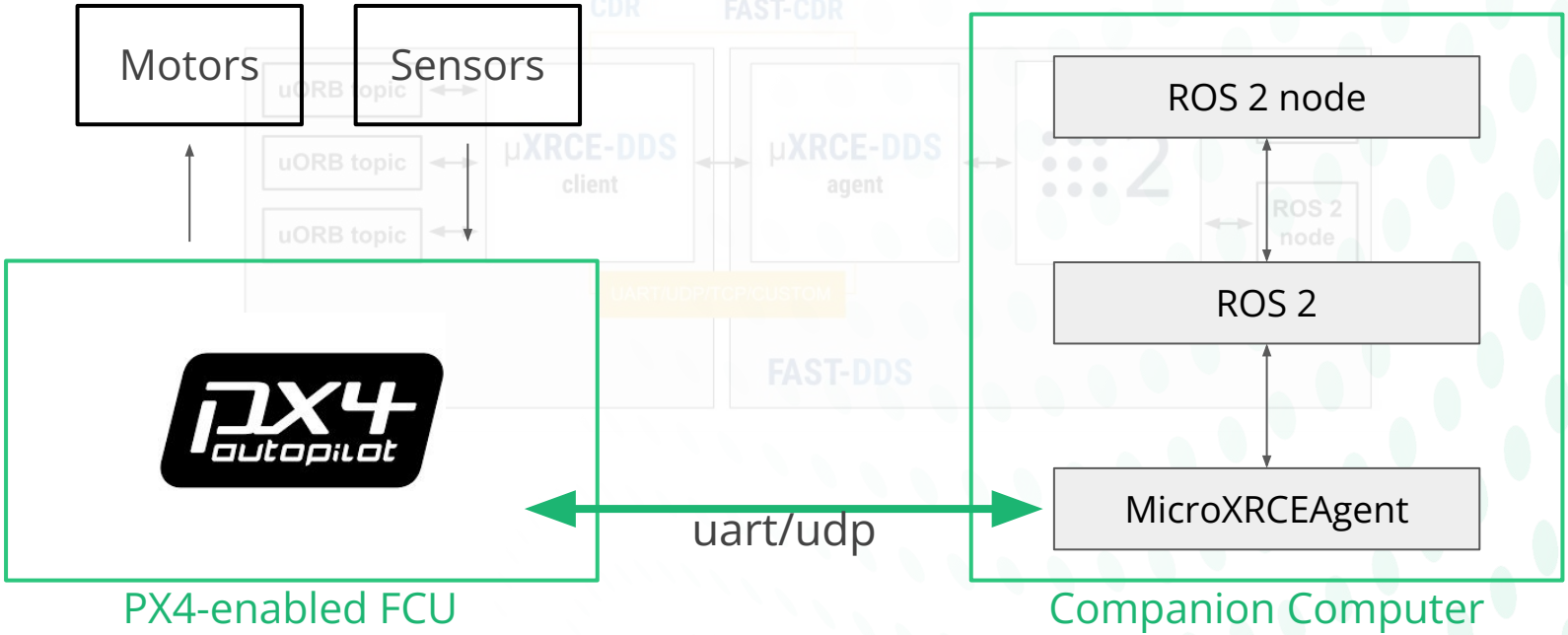
- Set `HEADLESS=1` if you don't want GUI to run
- To leverage GPU acceleration, if available, refer to https://docs.px4.io/main/en/simulation/gazebo_container_gui
- From inside the container, install the workspace dependencies and build the workspace

```
apt update
rosdep install -y -i --from-paths src --skip-keys OpenCV
source /opt/ros/jazzy/setup.bash
colcon build --symlink-install
```

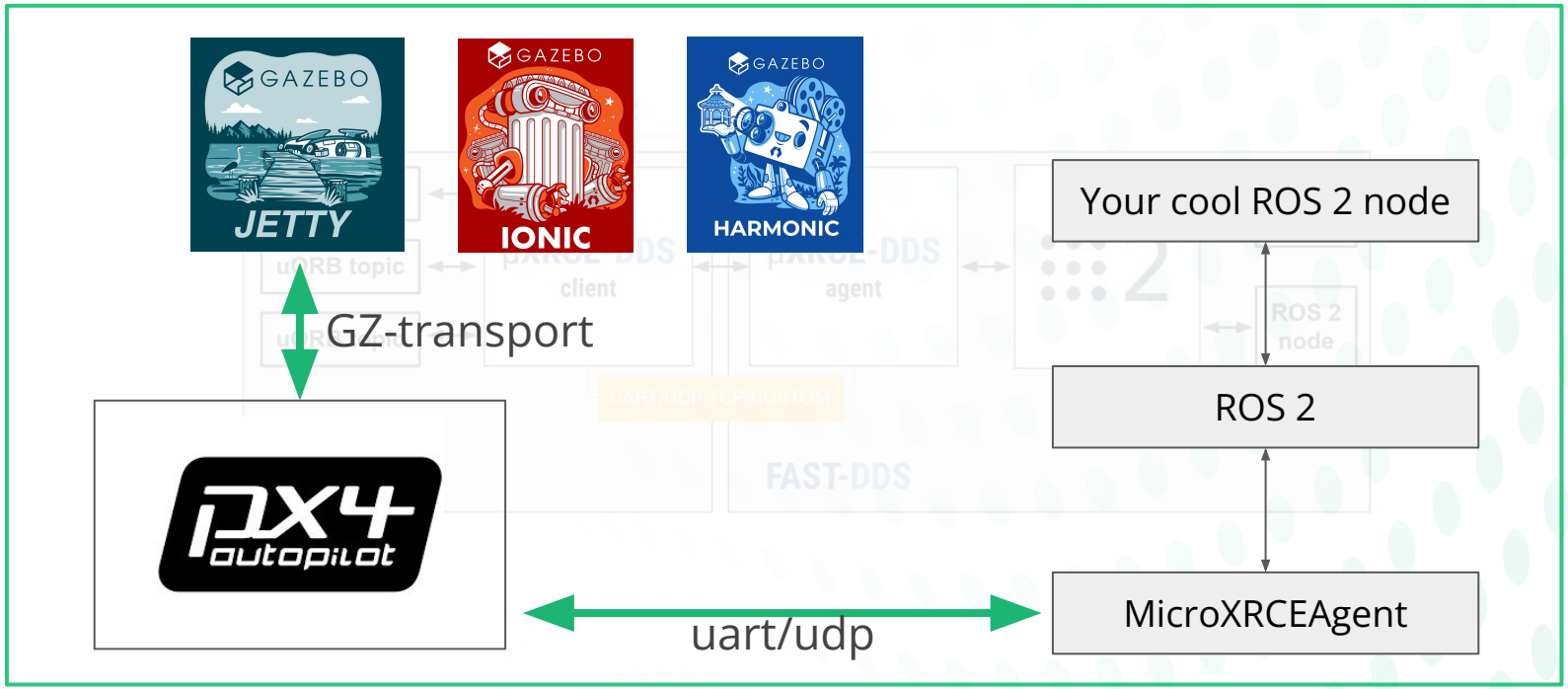
PX4 - ROS 2 interaction



PX4 - ROS 2 interaction



PX4 - ROS 2 interaction in Simulation



PX4 - ROS 2 interaction in Simulation



ROS - GZ

- ros_gz
- px4_msgs
- px4-ros2-interface-lib
- Micro-XRCE-DDS-Agent



GZ-transport

QGroundControl

Micro XRCE DDS

- px4_roscon_workshop
- Offboard demo
- Custom mode demo
- Aruco marker detector
- Teloperation
- Precision landing
- Formation control

PX4 - Gz Simulation startup

- Starting Gazebo and PX4: `make px4_sitl gz_{model}_{world}`

  E.g. `make px4_sitl gz_x500`

- Flexible configuration: https://docs.px4.io/main/en/sim_gazebo_gz/#usage-configuration-options

- Starting PX4 and gazebo from ros: `px4_roscon_workshop/launch/px4.launch.py`

References:

- https://github.com/PX4/PX4-gazebo-models
- https://docs.px4.io/main/en/sim_gazebo_gz/

Linking Gazebo and PX4 to ROS 2

- GZ => ROS 2 connection

```
ros2 run ros_gz_bridge parameter_bridge
/clock@rosgraph_msgs/msg/Clock[gz.msgs.Clock
```

- PX4 <=> ROS 2 connection

```
MicroXRCEAgent udp4 -p 8888
```

- Inspecting PX4 topics from ROS 2

```
ros2 topic list
ros2 topic echo /fmu/out/vehicle_status_v1
```

- Getting TF data from PX4

```
ros2 run px4_tf px4_tf_publisher --ros-args -p
use_sim_time:=true
```

- Running Foxglove bridge

```
ros2 run foxglove_bridge foxglove_bridge --ros-args -p
use_sim_time:=true
```

Prerequisites - environment setup - architecture overview

Now you know how

- ✓ Set up the simulation environment
- ✓ Fly a drone in Gazebo without ROS 2
- ✓ Attach ROS 2 to the simulation
- ✓ Echo PX4 topics in ROS 2
- ✓ Visualize PX4 data with Foxglove

Quick Break

Coming Next: Control Pipelines

Aerial Robotics Meetup, tonight!

Tonight, 8:00 to 11:00 PM
CRAFT Beer Market, 1 Adelaide St E (short walk from the venue)
Sponsored by Dronecode. **Food and drinks are on us.**

Talk shop on control architectures, state estimation, and open source flight stacks with the people who actually maintain them.



Control Pipelines (Patrik)

- Understanding how high-level control can be moved from the flight controller (PX4) to the onboard computer (ROS 2).
- Exploring how PX4 and ROS 2 share responsibilities for stability, autonomy, and perception-driven behavior.
- Learning when and why to let ROS 2 take over command generation while PX4 ensures real-time safety and actuator control.

Why take control from the flight controller:

- To implement custom autonomy, AI, or perception-based navigation.
- To integrate external sensor data (LiDAR, cameras, etc.) not directly handled by PX4.
- To test new control strategies without flashing or modifying flight firmware.
- To coordinate multi-robot systems from a common ROS 2 framework.

Taking Control of the Flight Controller

Why do we want to do that?

- Enable **autonomous behavior** and **custom logic**
- Integrate with **ROS 2** nodes for perception & planning
- Test **new control strategies** safely in simulation
- Go beyond built-in flight modes

Risks involved

- Loss of **failsafe protections**
- Unstable flight if setpoints are wrong or too slow
- Direct control = **direct responsibility**
- Always test in **SITL first**

How Can We Take Control Using ROS 2?

Offboard Mode — the *classic approach*

- Built around **MAVLink**, not ROS 2
- Works in **C++ or Python**
- Great for **quick testing**
- *Old method — not ROS 2-native*

https://docs.px4.io/main/en/ros2/offboard_control.html

PX4 ROS 2 Interface (External Modes)

- Uses `px4_ros2::ModeBase` → **direct PX4–ROS 2 bridge**
- C++, requires **more experience** (Python binders are available)
- Supports **complex behaviors and autonomy logic**
- Integrates **PX4 safety + lifecycle hooks**
- *Modern, flexible, ROS 2-native control*

https://docs.px4.io/main/en/ros2/px4_ros2_control_interface.html

What's New

Python bindings — prototype modes in Python, not just C++ (great for teaching, rapid iteration) *(verify exact package/API name on your slide)*

Rover & VTOL support — setpoint types beyond multicopter

Direct actuator control — `DirectActuatorsSetpointType` and `PeripheralActuatorControls` for motors, servos, and payload actuators

Same registration/lifecycle model across all of the above

https://docs.px4.io/main/en/ros2/px4_ros2_control_interface

> **TIP**
>
> If you want to use Python, check out the examples in the repository. Not all classes have Python bindings yet — the supported bindings are here. You are welcome to add and contribute missing classes.

What's New

Let’s try the Python example

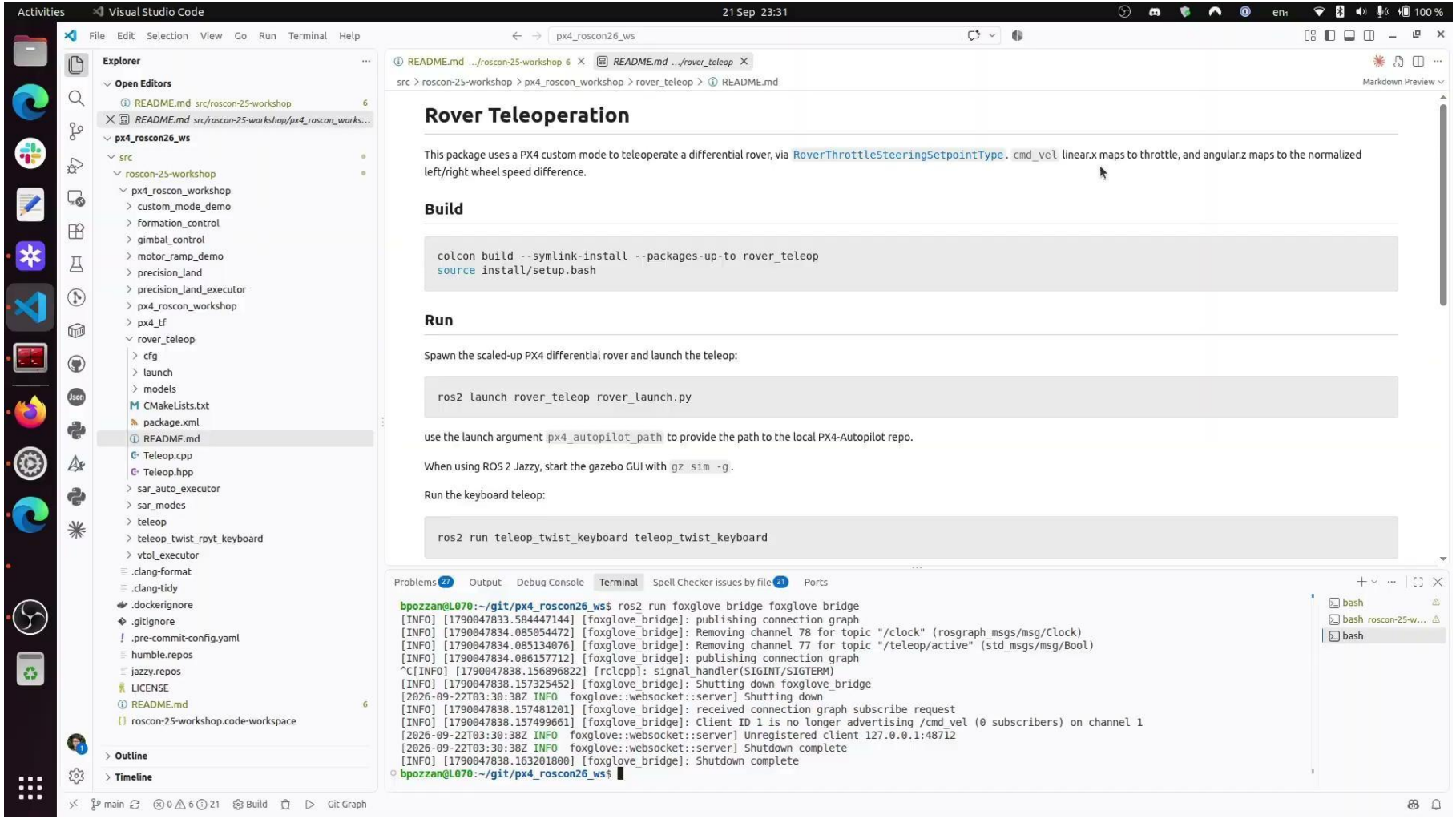
examples/python/modes/goto/example_mode_goto_py

ros2 run example_mode_goto_py goto

Let’s try the rover teleop example

ros2 launch rover_teleop rover_launch.py

ros2 run teleop_twist_keyboard teleop_twist_keyboard


Activities
Visual Studio Code
21 Sep 23:31
100 %
File Edit Selection View Go Run Terminal Help
px4_roscon26_ws
Explorer
Open Editors
README.md src/roscon-25-workshop 6
README.md src/roscon-25-workshop/px4_roscon_works...
px4_roscon26_ws
src
roscon-25-workshop
px4_roscon_workshop
custom_mode_demo
formation_control
gimbal_control
motor_ramp_demo
precision_land
precision_land_executor
px4_roscon_workshop
px4_tf
rover_teleop
cfg
launch
models
CMakeLists.txt
package.xml
README.md
Teleop.cpp
Teleop.hpp
sar_auto_executor
sar_modes
teleop
teleop_twist_rpyt_keyboard
vtol_executor
.clang-format
.clang-tidy
.dockerignore
.gitignore
.pre-commit-config.yaml
humble.repos
jazzy.repos
LICENSE
README.md 6
roscon-25-workshop.code-workspace
Outline
Timeline
README.md .../roscon-25-workshop 6
README.md .../rover_teleop
src > roscon-25-workshop > px4_roscon_workshop > rover_teleop > README.md
Markdown Preview
Rover Teleoperation
This package uses a PX4 custom mode to teleoperate a differential rover, via RoverThrottleSteeringSetpointType. cmd_vel linear.x maps to throttle, and angular.z maps to the normalized left/right wheel speed difference.
Build
colcon build --symlink-install --packages-up-to rover_teleop
source install/setup.bash
Run
Spawn the scaled-up PX4 differential rover and launch the teleop:
ros2 launch rover_teleop rover_launch.py
use the launch argument px4_autopilot_path to provide the path to the local PX4-Autopilot repo.
When using ROS 2 Jazzy, start the gazebo GUI with gz sim -g.
Run the keyboard teleop:
ros2 run teleop_twist_keyboard teleop_twist_keyboard
Problems 27 Output Debug Console Terminal Spell Checker issues by file 21 Ports
bpozzan@L070:~/git/px4_roscon26_ws$ ros2 run foxglove_bridge foxglove_bridge
[INFO] [1790047833.584447144] [foxglove_bridge]: publishing connection graph
[INFO] [1790047834.085054472] [foxglove_bridge]: Removing channel 78 for topic "/clock" (rosgraph_msgs/msg/Clock)
[INFO] [1790047834.085134076] [foxglove_bridge]: Removing channel 77 for topic "/teleop/active" (std_msgs/msg/Bool)
[INFO] [1790047834.086157712] [foxglove_bridge]: publishing connection graph
^C[INFO] [1790047838.156896822] [rclcpp]: signal_handler(SIGINT/SIGTERM)
[INFO] [1790047838.157325452] [foxglove_bridge]: Shutting down foxglove_bridge
[2026-09-22T03:30:38Z INFO foxglove::websocket::server] Shutting down
[INFO] [1790047838.157481201] [foxglove_bridge]: received connection graph subscribe request
[INFO] [1790047838.157499661] [foxglove_bridge]: Client ID 1 is no longer advertising /cmd_vel (0 subscribers) on channel 1
[2026-09-22T03:30:38Z INFO foxglove::websocket::server] Unregistered client 127.0.0.1:48712
[2026-09-22T03:30:38Z INFO foxglove::websocket::server] Shutdown complete
[INFO] [1790047838.163201800] [foxglove_bridge]: Shutdown complete
bpozzan@L070:~/git/px4_roscon26_ws$
bash
bash roscon-25-w...
bash
main
0 6 21
Build
Git Graph

Custom Modes: Mode vs Mode Executor

Mode (px4_ros2::ModeBase)

Defines **how the vehicle behaves** during a custom flight mode.

- Inherits from `px4_ros2::ModeBase`
- Implements:
  - `onActivate()` / `onDeactivate()`
  - `updateSetpoint()` → generates control setpoints
- Uses PX4 objects (e.g. `ManualControlInput`, `RatesSetpointType`)
- Registered with `mode->doRegister()`
- Focus: **control logic**, what the drone *does right now*

Think of this as the "brain" for a single flight mode.

Mode Executor (px4_ros2::ModeExecutorBase)

Coordinates **multiple modes or actions** into a higher-level behavior.

- Inherits from `px4_ros2::ModeExecutorBase`
- Owns one Mode (`MyMode`)
- Defines **states** → e.g. `TakingOff → MyMode → RTL`
- Uses helper actions: `takeoff()`, `scheduleMode()`, `rtl()`
- Registered via `executor->doRegister()`
- Focus: **mission flow**, *when and in what order* things happen

Think of this as the "mission director" controlling the brains.

Concept	Purpose	Example
Mode	Implements a specific behavior	Manual, Teleop, Offboard-like
ModeExecutor	Sequences modes/actions	Takeoff → Custom → Land

Custom Mode example

Node setup

- Uses `NodeWithModeExecutor<CustomModeExecutor, CustomWaypoints, CustomYaw>`
- Launches a ROS 2 node with two custom modes and one executor

Custom Modes

- **CustomWaypoints**
  - Generates and publishes position setpoints
  - Drone moves through a list of waypoints
  - Calls `completed()` when all waypoints reached
- **CustomYaw**
  - Commands slow yaw rotation setpoints
  - Tracks total rotation angle
  - Finishes after one full 360° turn

Mode Executor

- Implements a small state machine:
  Takeoff → CustomWaypoints → CustomYaw → Land → WaitUntilDisarmed
- Calls high-level actions (`takeoff()`, `land()`, `waitUntilDisarmed()`)
- Uses `scheduleMode()` to run each custom mode in sequence

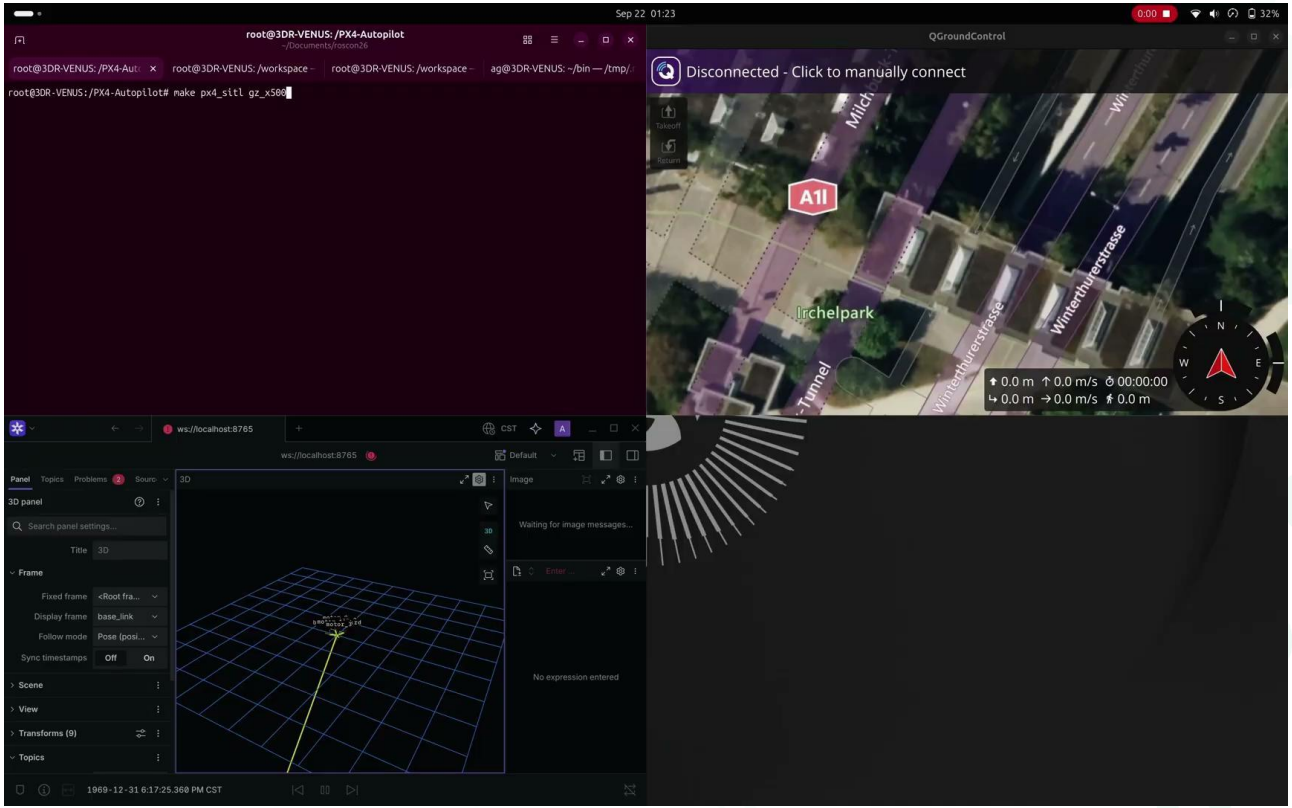
Control Flow

- Each step triggers a callback when done
- On success → moves to the next state
- Logs status at each transition

Takeaway

- Combines multiple **ROS 2-native modes** into one **autonomous flow**
- Clean separation:
  - **Modes** → what to do
  - **Executor** → when to do it
- No MAVLink commands needed directly
- Robust, modular, and fully **extensible** for new behaviors

Custom Mode Video



Custom mode exercises

Problems

1. GOTO/Trajectory Setpoint replacement
2. Explore what happens if you want to change the Mode order

Break (30 min)

Coming Next: Mode Executors and Multi-Vehicle Control

Mode Executors

- VTOL = two regimes in one airframe: multicopter (MC) hover and fixed-wing (FW) cruise, plus the transitions between them
- Pick setpoint types — MC setpoints in hover, forward-flight setpoints in cruise; don't reuse an MC-only type in FW
- MC takeoff → transition to FW → cruise / run custom FW mode → transition back to MC → land

Custom Modes: Mode vs Mode Executor

Mode (px4_ros2::ModeBase)

Defines **how the vehicle behaves** during a custom flight mode.

- Inherits from `px4_ros2::ModeBase`
- Implements:
  - `onActivate()` / `onDeactivate()`
  - `updateSetpoint()` → generates control setpoints
- Uses PX4 objects (e.g. `ManualControlInput`, `RatesSetpointType`)
- Registered with `mode->doRegister()`
- Focus: **control logic**, what the drone *does right now*

Think of this as the "brain" for a single flight mode.

Mode Executor (px4_ros2::ModeExecutorBase)

Coordinates **multiple modes or actions** into a higher-level behavior.

- Inherits from `px4_ros2::ModeExecutorBase`
- Owns one Mode (`MyMode`)
- Defines **states** → e.g. `TakingOff → MyMode → RTL`
- Uses helper actions: `takeoff()`, `scheduleMode()`, `rtl()`
- Registered via `executor->doRegister()`
- Focus: **mission flow**, *when and in what order* things happen

Think of this as the "mission director" controlling the brains.

Concept	Purpose	Example
Mode	Implements a specific behavior	Manual, Teleop, Offboard-like
ModeExecutor	Sequences modes/actions	Takeoff → Custom → Land

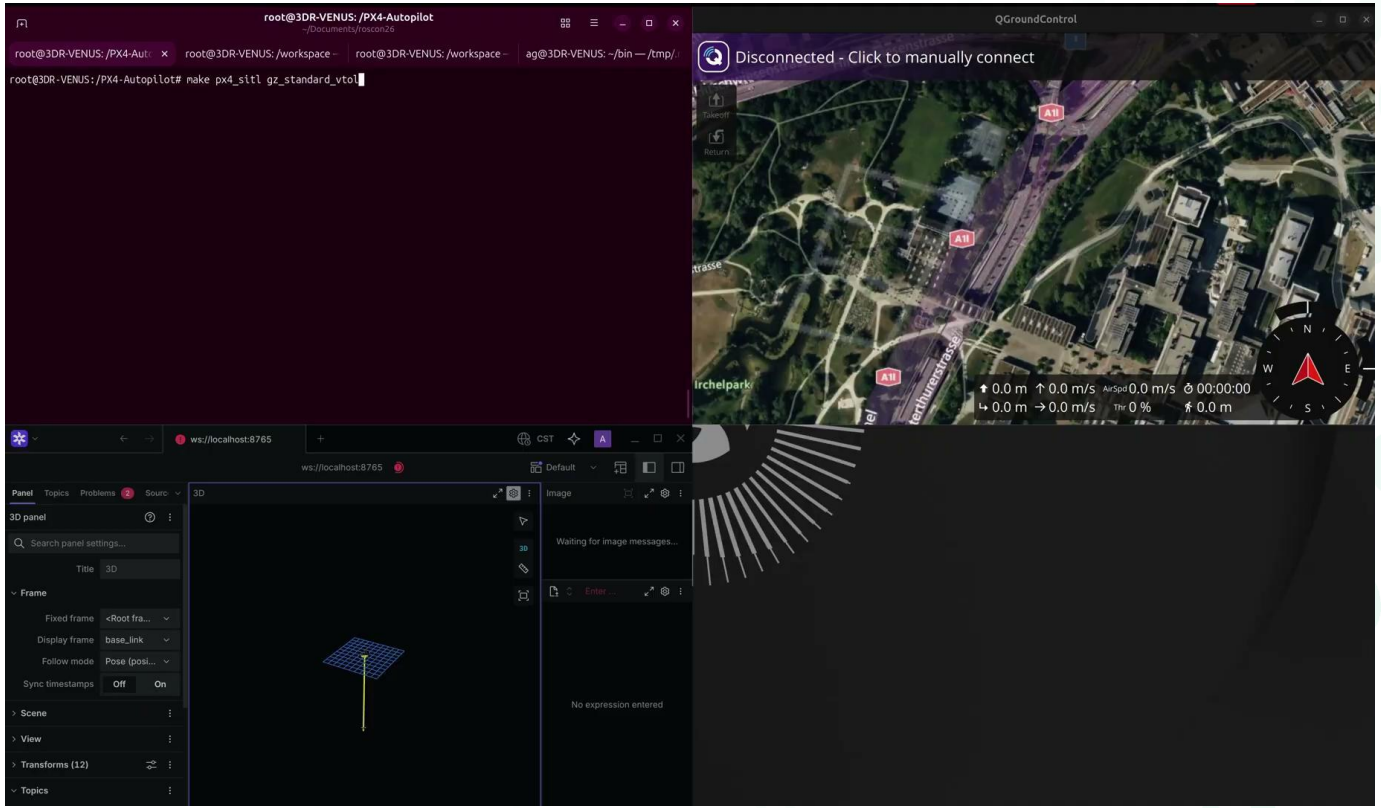
VTOL executor exercise

workspaces/px4_roscon26_ws/src/px4-roscon-workshop/px4_roscon_workshop/vtol_executor

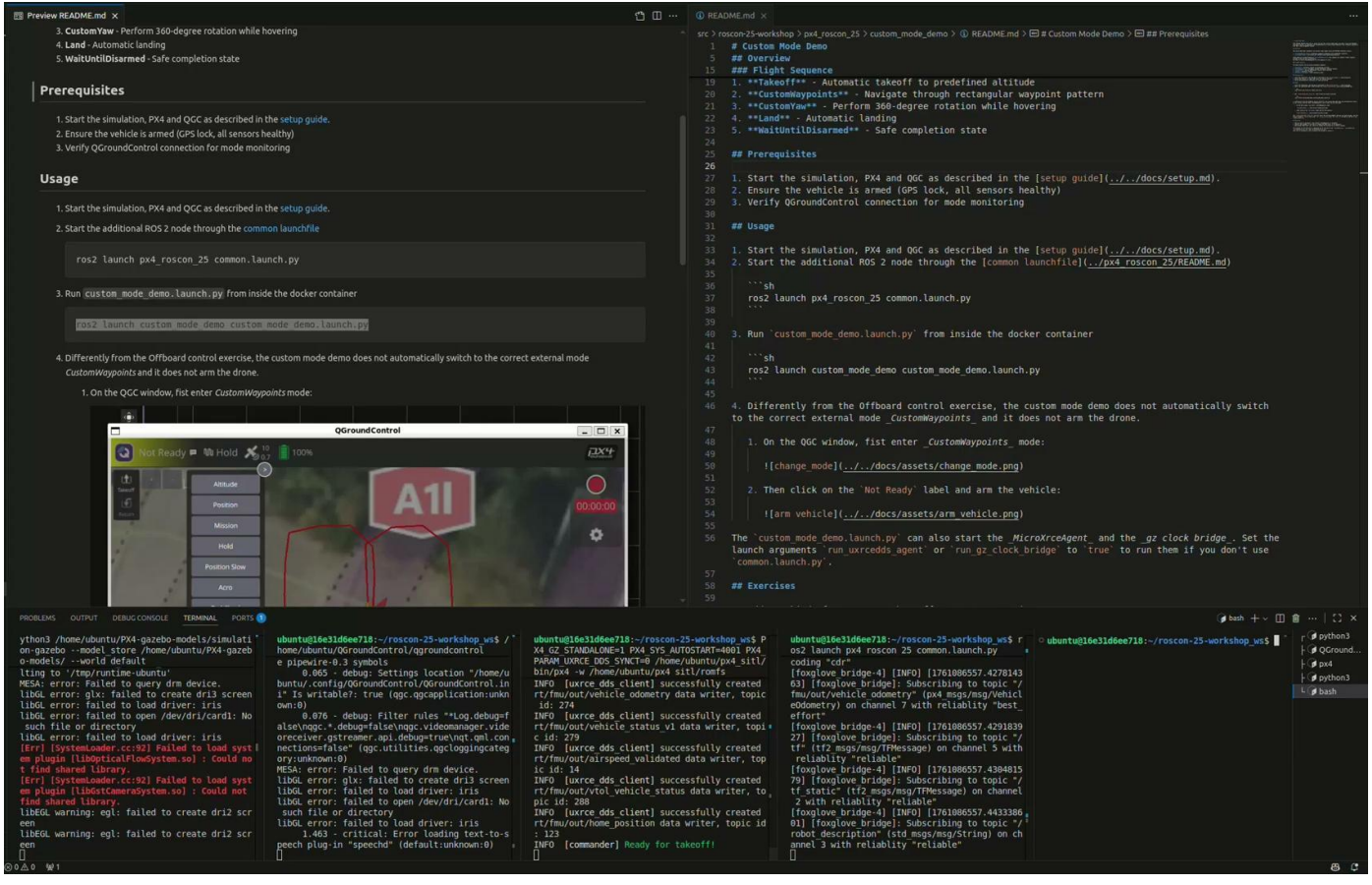
Problems

1. Change land to RTL

VTOL executor video



Quick video - MC mode executor



Multi-Vehicle Control (Patrik & Beniamino)

- Each PX4 instance runs its own uXRCE-DDS client with a unique instance ID / topic namespace; one agent bridges them all into the ROS 2 graph
- Run one executor node per vehicle, scoped to that vehicle's ROS 2 namespace (e.g. /px4_1, /px4_2) — the library binds to the matching instance
- Add a coordinator/orchestrator node above the executors for fleet logic: task allocation, formation, synchronized takeoff, deconfliction
- Executors handle per-vehicle safety and sequencing; the orchestrator handles inter-vehicle decisions — clean separation of concerns
- SITL: launch N PX4 instances with distinct instance IDs + DDS namespaces; all appear in QGC and Gazebo together

Multi-Vehicle Control exercise

workspaces/px4_roscon26_ws/src/px4-roscon-workshop/px4_roscon_workshop/formation_control

Problem

1. Formation Control: make it an executor that auto takes off and then transit to formation control (this could actually be how we present formation control).

Lunch Break (60 min)

Coming Next: Perception & Applications

Aerial Robotics Meetup, tonight!

Tonight, 8:00 to 11:00 PM
CRAFT Beer Market, 1 Adelaide St E (short walk from the venue)
Sponsored by Dronecode. **Food and drinks are on us.**

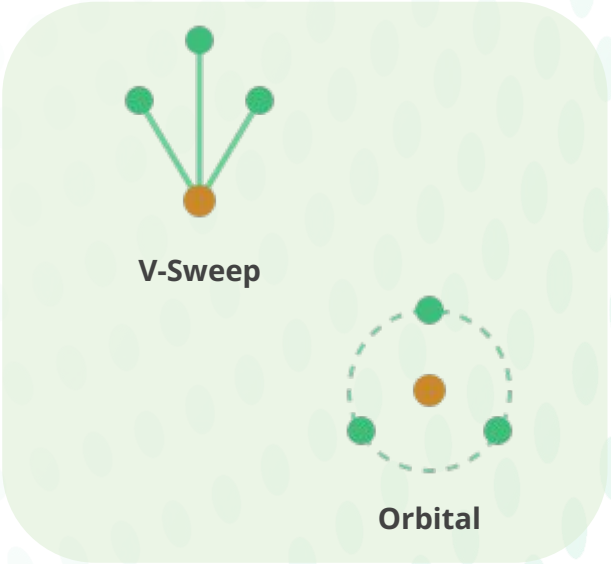
Talk shop on control architectures, state estimation, and open source flight stacks with the people who actually maintain them.



Application Example: Search And Rescue (SAR) modes

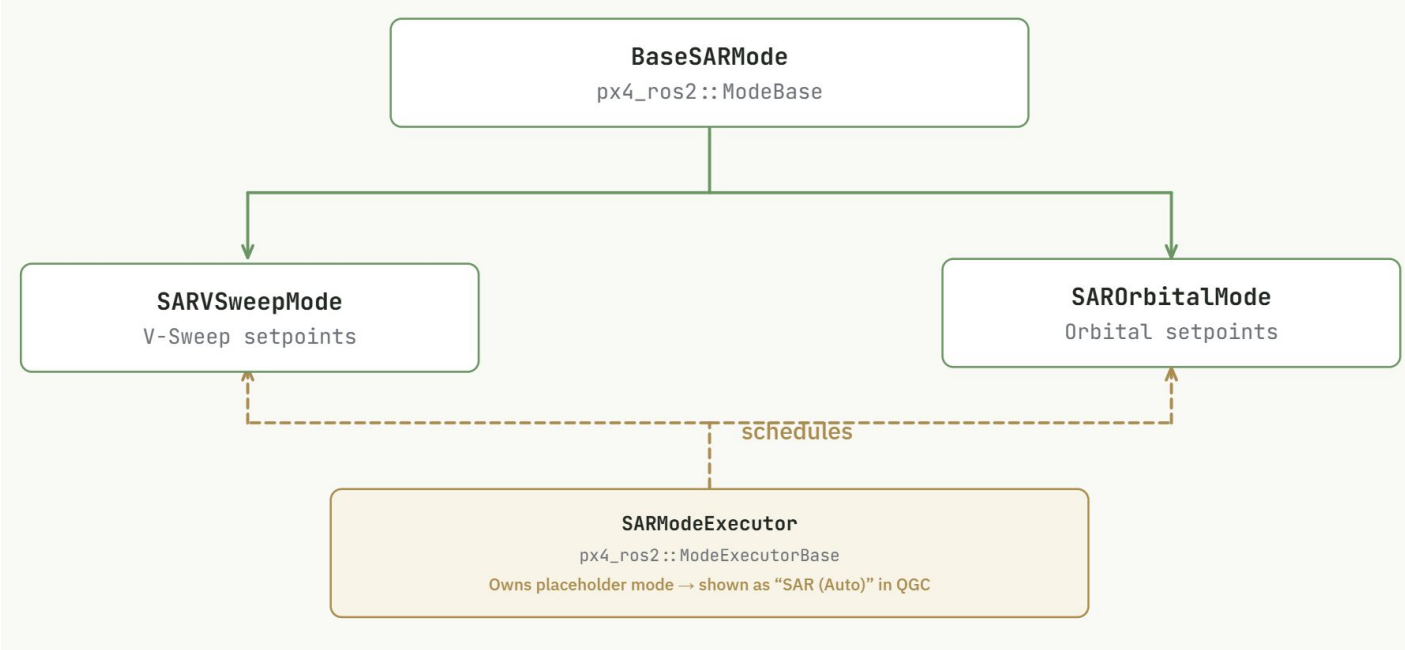
Scenario

- SAR swarm: a rover is the active rescuer, drones extend its "instruments and perspective."
- **V-Sweep** = wide aerial scan while it moves
- **Orbital** = 360° when it stops to investigate



Didactic example only, not a search-and-rescue readiness demo

SAR modes architecture + SAR executor (Auto)



Solid = inherits & Dashed = executor schedules
(holds references, not a subclass)

SAR Modes vs Formation Control

SAR Modes

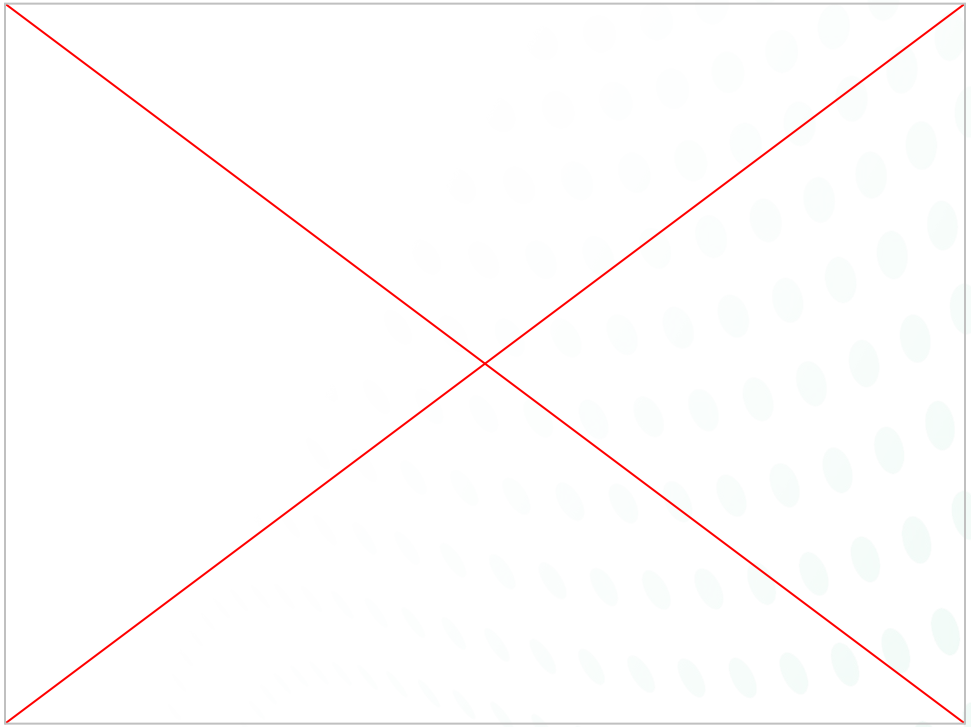
- Target relative: every drone tracks one external target (rover)
- Reads a single rover pose (*/rover/pose*)
- Formation: V-Sweep wedge or Orbital circle
- No inter-drone awareness (yet)

Formation Control

- Peer-to-peer: every drone tracks its neighbors
- Reads TF transforms between vehicles
- Formation: Fixed
- Spring-based separation controller

Shared infrastructure, different brain

Quick Video



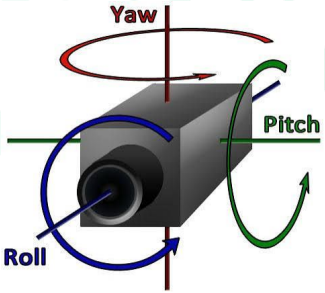
Exercises

1. **sar_modes:**
    a. Verify the formation actually re-arranges when you flip modes manually in QGC
    b. Build a third SAR mode of your own
2. **sar_auto_executor:**
    a. Verify in SAR (Auto) that QGC switches on its own as the rover moves, with the console open alongside to watch the executor’s state transitions
    b. Build either a take-off step for the executor, or take-off plus a state driven by rover’s behavior (e.g. stays still for *t* seconds)

Gimbal control

- Three axis gimbal system
- yaw -> roll -> pitch
- Simulated in gazebo by the `vtol_gimbal` model
- Low level controlled in PX4 SITL by the gz_bridge
  - Supports MAVLink Gimbal V2 protocol
  - Support direct control over `gimbal_controls` uorb topic
- We will control the gimbal from ROS 2 directly publishing on `/fmu/in/gimbal_controls`

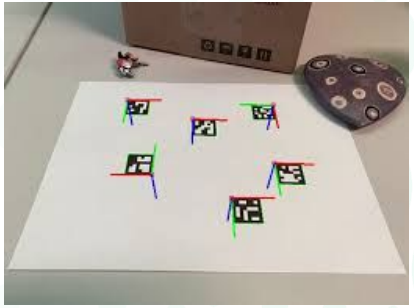




Perception & Applications

- Build a perception pipeline for ArUco marker landing and LiDAR mapping
- Demo vision-guided precision landing
- Architecture diagrams and code snippets

Aruco Marker detection



What is an ArUco Marker?

- A **binary square fiducial marker** used for camera pose estimation.
- Encodes a **unique ID** that can be easily detected in images.
- Used to determine **position and orientation** of objects relative to a camera.

Why Use ArUco Markers

- **Fast and lightweight** — works in real time on CPUs.
- **Highly reliable** under varied lighting and partial occlusion.
- **No training data required**, unlike AI-based object detection.
- **Deterministic performance** — consistent and easy to debug.

Why OpenCV (vs. AI)

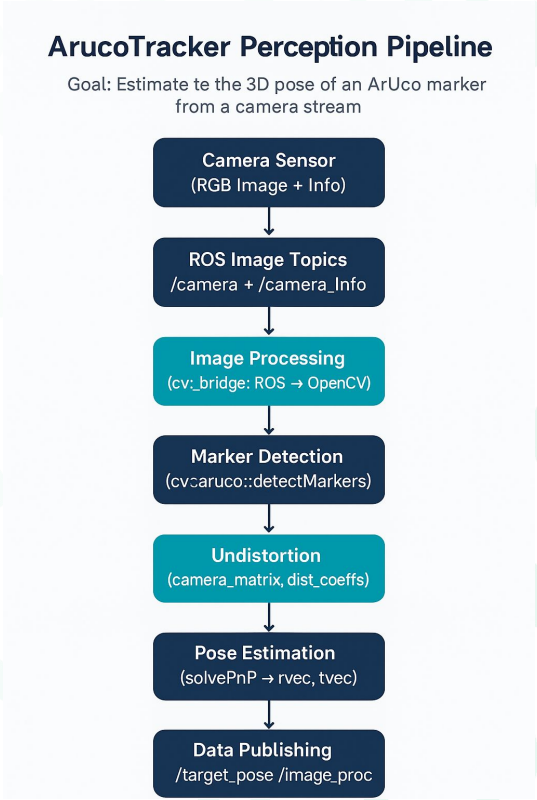
- **Open-source and widely supported** in ROS and C++.
- **Low computational cost** — ideal for embedded and robotics systems.
- **Easy integration** with camera calibration and PnP solvers.
- **Predictable results** without needing neural network inference.

3D Coordinate Frame Visualization

- The **3D axes (X, Y, Z)** show the marker's orientation in space.
- Allows intuitive visualization of **camera–marker geometry**.
- Essential for **pose estimation, control, and navigation** tasks.
- Helps debug spatial alignment in simulation and real-world setups.

Aruco Marker detection

Step	Description	Output
Image Acquisition	Subscribes to /camera and /camera_info	Raw RGB + calibration data
Marker Detection	Detects marker corners and IDs using OpenCV ArUco.	Marker corner pixel coords
Undistortion	Corrects corner coordinates using camera intrinsics.	Undistorted 2D points
Pose Estimation	Solves PnP to get marker position and orientation.	tvec, rvec
Pose Conversion	Converts rotation to quaternion for ROS compatibility.	PoseStamped message
Annotation & Publish	Draws axes, overlays text, publishes processed image and target pose.	/image_proc, /target_pose



Aruco Marker exercise

Implement a ROS 2 subscription that verifies the size of a detected ArUco marker using the distance measured by a rangefinder.

In this exercise, we estimate the real-world size of a detected ArUco marker using only the camera image, its intrinsic parameters, and the vehicle's measured altitude. Show this estimated marker size on the annotated image.

Hint: https://docs.px4.io/main/en/msg_docs/VehicleLocalPosition

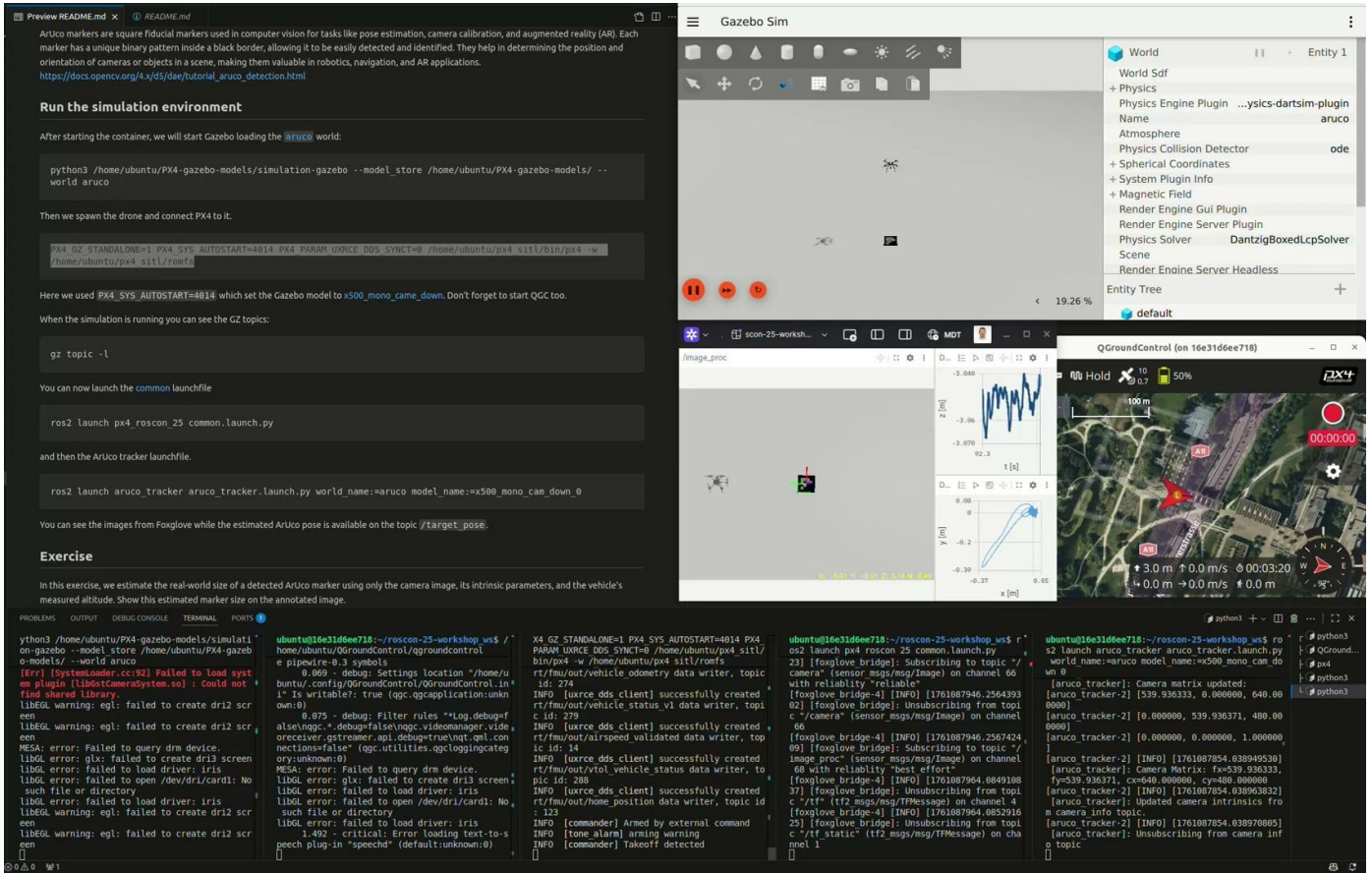
The apparent (measured) size of the marker in the image depends on the camera's focal length and the distance to the target.
We can estimate the **real-world marker size** as:

$$L_{est} = \frac{p}{f} \cdot d$$

where:

- L_{est} — estimated marker size in meters
- p — marker width in pixels (distance between two adjacent corners)
- f — camera focal length in pixels (from the camera matrix)
- d — distance from the camera to the ground, provided by PX4 (`dist_bottom`)

Quick video


Preview README.md
README.md
ArUco markers are square fiducial markers used in computer vision for tasks like pose estimation, camera calibration, and augmented reality (AR). Each marker has a unique binary pattern inside a black border, allowing it to be easily detected and identified. They help in determining the position and orientation of cameras or objects in a scene, making them valuable in robotics, navigation, and AR applications.
https://docs.opencv.org/4.x/d5/dae/tutorial_aruco_detection.html
Run the simulation environment
After starting the container, we will start Gazebo loading the aruco world:
python3 /home/ubuntu/PX4-gazebo-models/simulation-gazebo --model_store /home/ubuntu/PX4-gazebo-models/ --world aruco
Then we spawn the drone and connect PX4 to it.
PX4_GZ_STANDALONE=1 PX4_SYS_AUTOSTART=4014 PX4_PARAM_UXRCE_DDS_SYNCT=0 /home/ubuntu/px4_sitl/bin/px4 -w /home/ubuntu/px4_sitl/romfs
Here we used PX4_SYS_AUTOSTART=4014 which set the Gazebo model to x500_mono_came_down. Don't forget to start QGC too.
When the simulation is running you can see the GZ topics:
gz topic -l
You can now launch the common launchfile
ros2 launch px4_roscon_25 common.launch.py
and then the ArUco tracker launchfile.
ros2 launch aruco_tracker aruco_tracker.launch.py world_name:=aruco model_name:=x500_mono_cam_down_0
You can see the images from Foxglove while the estimated ArUco pose is available on the topic /target_pose.
Exercise
In this exercise, we estimate the real-world size of a detected ArUco marker using only the camera image, its intrinsic parameters, and the vehicle's measured altitude. Show this estimated marker size on the annotated image.
Gazebo Sim
World
Entity 1
World Sdf
Physics
Physics Engine Plugin
...ysics-dartsim-plugin
Name
aruco
Atmosphere
Physics Collision Detector
ode
Spherical Coordinates
System Plugin Info
Magnetic Field
Render Engine Gui Plugin
Render Engine Server Plugin
Physics Solver
DantzigBoxedLcpSolver
Scene
Render Engine Server Headless
Entity Tree
default
19.26 %
QGroundControl (on 16e31d6ee718)
Hold
50%
00:00:00
3.0 m ↑0.0 m/s 00:03:20
0.0 m →0.0 m/s 0.0 m
PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
ubuntu@16e31d6ee718:~/roscon-25-workshop_ws$
ubuntu@16e31d6ee718:~/roscon-25-workshop_ws$ ros2 launch aruco_tracker aruco_tracker_launch.py world_name:=aruco model_name:=x500_mono_cam_down_0
[aruco_tracker]: Camera matrix updated:
[aruco_tracker-2] [539.936333, 0.000000, 640.000000]
[aruco_tracker-2] [0.000000, 539.936371, 480.000000]
[aruco_tracker-2] [0.000000, 0.000000, 1.000000]
[aruco_tracker-2] [INFO] [1761087854.038949530] [aruco_tracker]: Camera Matrix: fx=539.936333, fy=539.936371, cx=640.000000, cy=480.000000
[aruco_tracker-2] [INFO] [1761087854.038963832] [aruco_tracker]: Updated camera intrinsics from camera_info topic.
[aruco_tracker-2] [INFO] [1761087854.038970805] [aruco_tracker]: Unsubscribing from camera_info topic

Teleoperation & Lidar Scanning

Teleoperation Overview

- Direct control of the drone via joystick or keyboard (Twist commands).
- Integration with PX4 through ROS 2 topics (/cmd_vel → PX4 setpoints).
- Supports manual navigation in indoor/outdoor environments.
- Enables real-time response to operator inputs.
- Can be combined with autonomous routines for hybrid control.

LiDAR Scanning Overview

- Collects high-resolution 3D point cloud data of the environment.
- Provides obstacle detection and mapping capabilities.
- Uses ROS 2 sensor messages (sensor_msgs::PointCloud2) for integration.
- Supports real-time visualization in tools like RViz or Foxglove Studio.
- Useful for mapping, inspection, and SLAM experiments.

Teleoperation & Lidar Scanning

Mode with Executor

- **Takeoff state->TeleOperation->Land**

Teleoperation

- **Teleop_twist_keyboard**
- **ENU/NED conversion**
- **Holonomic control**

Lidar Scan

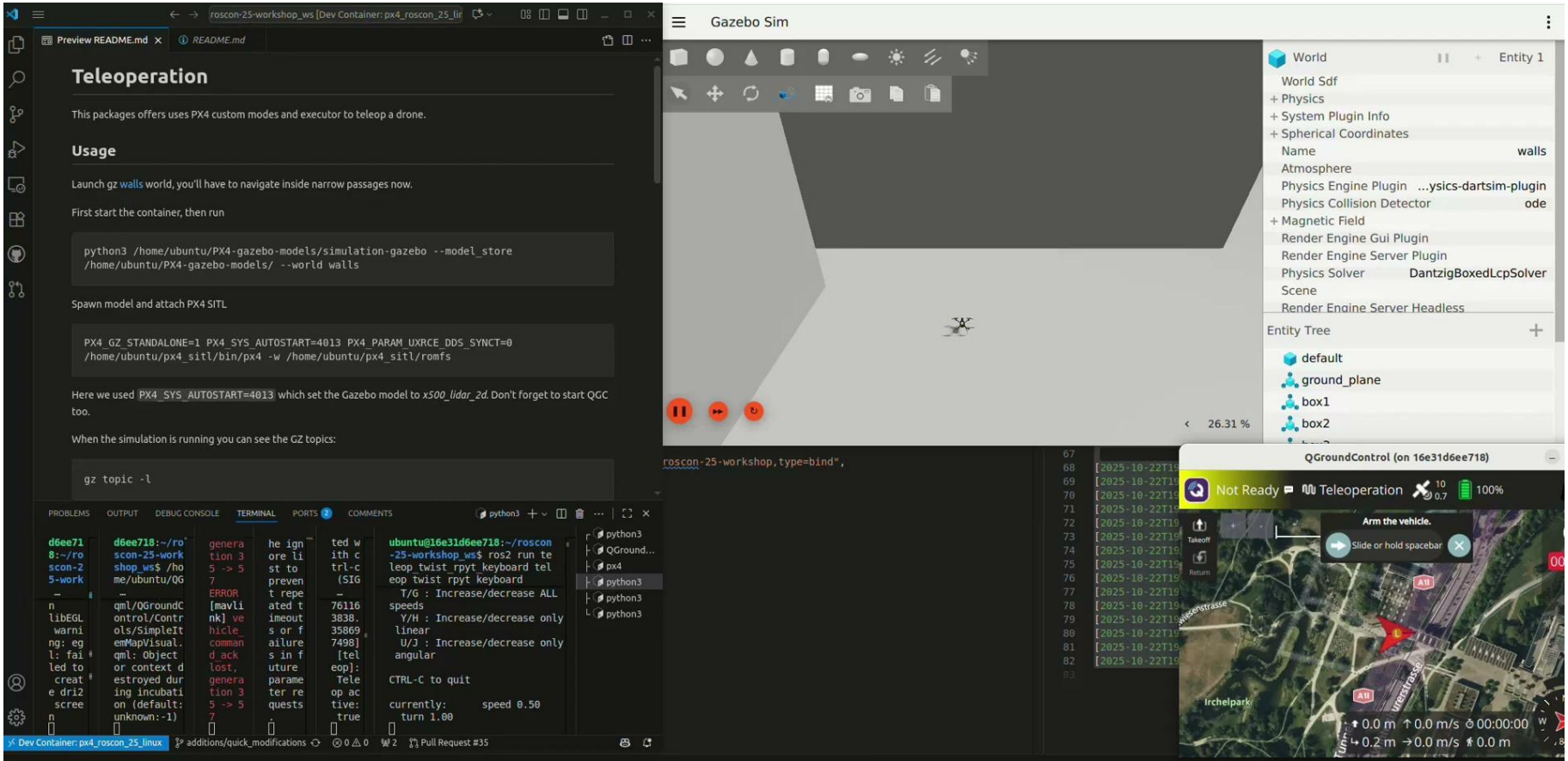
- **2d laserscan is mounted on the drone**
- **Live visualization on Foxglove**

Teleoperation & Lidar Scanning exercise

Navigate the drone to the target in Foxglove using teleoperation. Do **not** use the Gazebo simulation—rely solely on the LiDAR (laser scan) data for guidance.

Hint: Observe and record a few key waypoints along the path; these will be useful for a subsequent exercise.
https://docs.px4.io/main/en/msg_docs/VehicleLocalPosition

Quick video


Preview README.md
README.md
Teleoperation
This packages offers uses PX4 custom modes and executor to teleop a drone.
Usage
Launch gz walls world, you'll have to navigate inside narrow passages now.
First start the container, then run
python3 /home/ubuntu/PX4-gazebo-models/simulation-gazebo --model_store /home/ubuntu/PX4-gazebo-models/ --world walls
Spawn model and attach PX4 SITL
PX4_GZ_STANDALONE=1 PX4_SYS_AUTOSTART=4013 PX4_PARAM_UXRCE_DDS_SYNC=0 /home/ubuntu/px4_sitl/bin/px4 -w /home/ubuntu/px4_sitl/romfs
Here we used PX4_SYS_AUTOSTART=4013 which set the Gazebo model to x500_lidar_2d. Don't forget to start QGC too.
When the simulation is running you can see the GZ topics:
gz topic -l
Gazebo Sim
World
Entity 1
World Sdf
Physics
System Plugin Info
Spherical Coordinates
Name
walls
Atmosphere
Physics Engine Plugin
Physics Collision Detector
ode
Magnetic Field
Render Engine Gui Plugin
Render Engine Server Plugin
Physics Solver
DantzigBoxedLcpSolver
Scene
Render Engine Server Headless
Entity Tree
default
ground_plane
box1
box2
26.31 %
PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
COMMENTS
ubuntu@16e31d6ee718:~/roscon-25-workshop_ws$ ros2 run teleop_twist_rpyt_keyboard teleop_twist_rpyt_keyboard
T/G : Increase/decrease ALL speeds
Y/H : Increase/decrease only linear
U/J : Increase/decrease only angular
CTRL-C to quit
currently: speed 0.50 turn 1.00
QGroundControl (on 16e31d6ee718)
Not Ready
Teleoperation
100%
Arm the vehicle.
Slide or hold spacebar
Takeoff
Return
Irchelpark
Dev Container: px4_roscon_25_linux
additions/quick_modifications
Pull Request #35

Precision Land - Mode

Precision landing is the process of autonomously guiding a drone to land at a specific target (e.g., a marker or pad) with high accuracy, using onboard sensors (like cameras or LiDAR) to detect and track the target.

Key Features in this Implementation:

- Uses **Aruco markers** as landing targets.
- Performs **spiral search pattern** if the target is not immediately visible.
- Maintains **approach altitude**, then descends once target is acquired.
- XY motion uses a **velocity controller with proportional–integral (PI) gains**.
- Uses **land detection feedback** to confirm landing success.

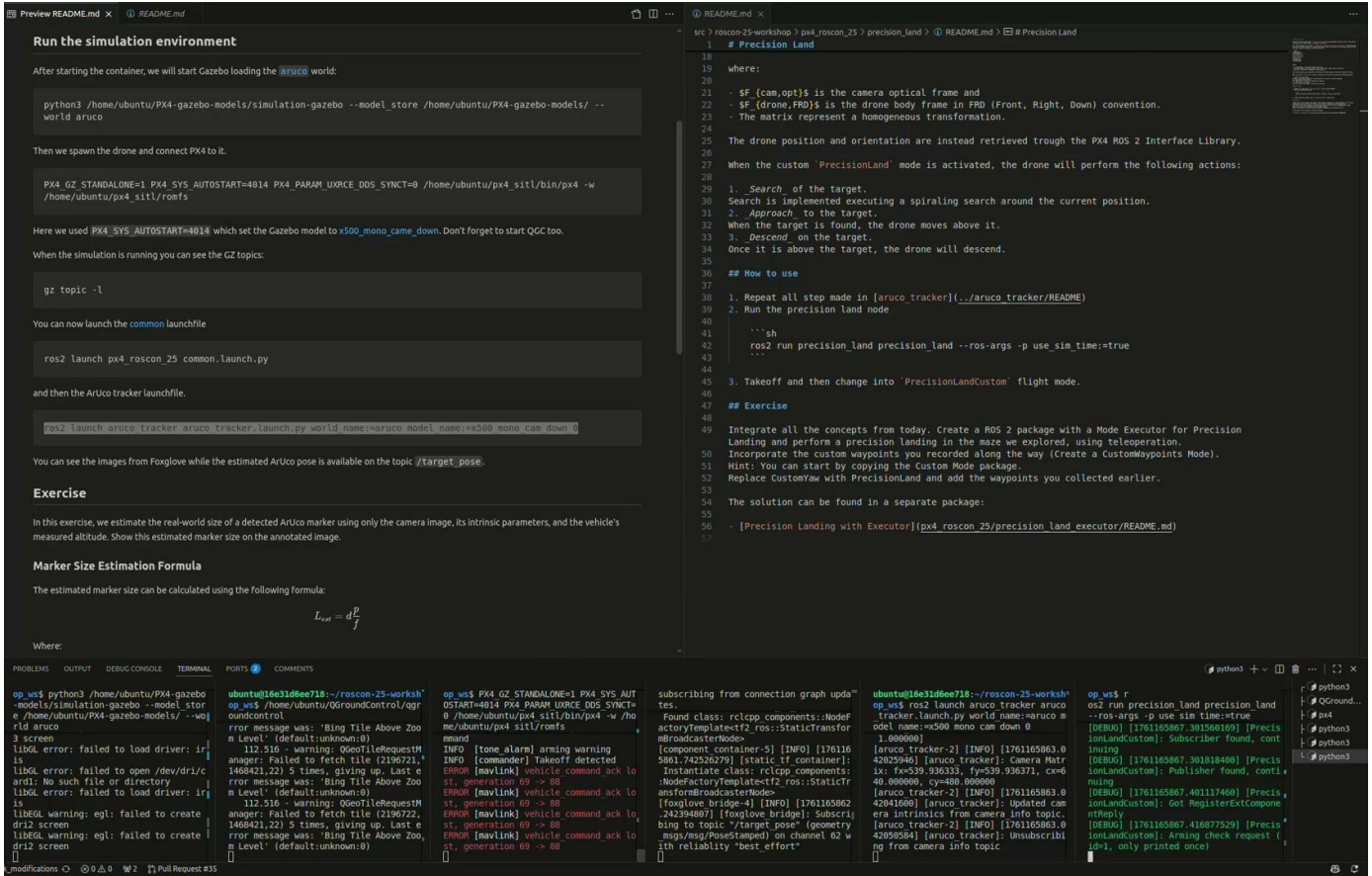
The drone must convert target measurements from the camera's frame (optical) to the drone's local NED (North-East-Down) frame.

Transform Steps in Code:

1. Convert Aruco tag pose from optical frame to NED frame using a rotation matrix.
2. Transform tag position relative to the **drone body frame**.
3. Transform to **world frame** by combining drone position & orientation (affine transform).

$$T_{\text{tag_world}} = T_{\text{drone_world}} \cdot T_{\text{camera_drone}} \cdot T_{\text{tag_camera}}$$

Quick video


Preview README.md
README.md
Run the simulation environment
After starting the container, we will start Gazebo loading the aruco world:
python3 /home/ubuntu/PX4-gazebo-models/simulation-gazebo --model_store /home/ubuntu/PX4-gazebo-models/ --world aruco
Then we spawn the drone and connect PX4 to it.
PX4_GZ_STANDALONE=1 PX4_SYS_AUTOSTART=4014 PX4_PARAM_UXRCE_DDS_SYNCT=0 /home/ubuntu/px4_sitl/bin/px4 -w /home/ubuntu/px4_sitl/romfs
Here we used PX4_SYS_AUTOSTART=4014 which set the Gazebo model to x500_mono_came_down. Don't forget to start QGC too.
When the simulation is running you can see the GZ topics:
gz topic -l
You can now launch the common launchfile
ros2 launch px4_roscon_25 common.launch.py
and then the ArUco tracker launchfile.
ros2 launch aruco_tracker aruco_tracker.launch.py world_name:=aruco model_name:=x500_mono_cam_down_0
You can see the images from Foxglove while the estimated ArUco pose is available on the topic /target_pose.
Exercise
In this exercise, we estimate the real-world size of a detected ArUco marker using only the camera image, its intrinsic parameters, and the vehicle's measured altitude. Show this estimated marker size on the annotated image.
Marker Size Estimation Formula
The estimated marker size can be calculated using the following formula:
$$L_{est} = d\frac{p}{f}$$
Where:
src > roscon-25-workshop > px4_roscon_25 > precision_land > README.md > # Precision Land
Precision Land
where:
- $F_{cam,opt}$ is the camera optical frame and
- $F_{drone,FRD}$ is the drone body frame in FRD (Front, Right, Down) convention.
- The matrix represent a homogeneous transformation.
The drone position and orientation are instead retrieved trough the PX4 ROS 2 Interface Library.
When the custom `PrecisionLand` mode is activated, the drone will perform the following actions:
1. _Search_ of the target.
Search is implemented executing a spiraling search around the current position.
2. _Approach_ to the target.
When the target is found, the drone moves above it.
3. _Descend_ on the target.
Once it is above the target, the drone will descend.
How to use
1. Repeat all step made in [aruco_tracker](../aruco_tracker/README)
2. Run the precision land node
```sh
ros2 run precision_land precision_land --ros-args -p use_sim_time:=true
```
3. Takeoff and then change into `PrecisionLandCustom` flight mode.
Exercise
Integrate all the concepts from today. Create a ROS 2 package with a Mode Executor for Precision Landing and perform a precision landing in the maze we explored, using teleoperation.
Incorporate the custom waypoints you recorded along the way (Create a CustomWaypoints Mode).
Hint: You can start by copying the Custom Mode package.
Replace CustomYaw with PrecisionLand and add the waypoints you collected earlier.
The solution can be found in a separate package:
- [Precision Landing with Executor](px4_roscon_25/precision_land_executor/README.md)
PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS 2
COMMENTS
op_ws$ python3 /home/ubuntu/PX4-gazebo-models/simulation-gazebo --model_store /home/ubuntu/PX4-gazebo-models/ --world aruco
3 screen
libGL error: failed to load driver: iris
libGL error: failed to open /dev/dri/card1: No such file or directory
libGL error: failed to load driver: iris
libEGL warning: egl: failed to create dri2 screen
libEGL warning: egl: failed to create dri2 screen
ubuntu@16e31d6ee718:~/roscon-25-workshop
op_ws$ /home/ubuntu/QGroundControl/qgroundcontrol
rror message was: 'Bing Tile Above Zoom Level' (default:unknown:0)
112.516 - warning: QGeoTileRequestManager: Failed to fetch tile (2196721,1468421,22) 5 times, giving up. Last error message was: 'Bing Tile Above Zoom Level' (default:unknown:0)
112.516 - warning: QGeoTileRequestManager: Failed to fetch tile (2196722,1468421,22) 5 times, giving up. Last error message was: 'Bing Tile Above Zoom Level' (default:unknown:0)
op_ws$ PX4_GZ_STANDALONE=1 PX4_SYS_AUTOSTART=4014 PX4_PARAM_UXRCE_DDS_SYNCT=0 /home/ubuntu/px4_sitl/bin/px4 -w /home/ubuntu/px4_sitl/romfs
mmand
INFO [tone_alarm] arming warning
INFO [commander] Takeoff detected
ERROR [mavlink] vehicle_command_ack lost, generation 69 -> 88
ERROR [mavlink] vehicle_command_ack lost, generation 69 -> 88
ERROR [mavlink] vehicle_command_ack lost, generation 69 -> 88
ERROR [mavlink] vehicle_command_ack lost, generation 69 -> 88
subscribing from connection graph updates.
Found class: rclcpp_components::NodeFactoryTemplate<tf2_ros::StaticTransformBroadcasterNode>
[component_container-5] [INFO] [1761165861.742526279] [static_tf_container]: Instantiate class: rclcpp_components::NodeFactoryTemplate<tf2_ros::StaticTransformBroadcasterNode>
[foxglove_bridge-4] [INFO] [1761165862.242394807] [foxglove_bridge]: Subscribing to topic "/target_pose" (geometry_msgs/msg/PoseStamped) on channel 62 with reliablity "best_effort"
ubuntu@16e31d6ee718:~/roscon-25-workshop
op_ws$ ros2 launch aruco_tracker aruco_tracker.launch.py world_name:=aruco model name:=x500 mono cam down 0
1.000000]
[aruco_tracker-2] [INFO] [1761165863.042025946] [aruco_tracker]: Camera Matrix: fx=539.936333, fy=539.936371, cx=640.000000, cy=480.000000
[aruco_tracker-2] [INFO] [1761165863.042041600] [aruco_tracker]: Updated camera intrinsics from camera info topic.
[aruco_tracker-2] [INFO] [1761165863.042050584] [aruco_tracker]: Unsubscribing from camera info topic
op_ws$ r
os2 run precision_land precision_land --ros-args -p use_sim_time:=true
[DEBUG] [1761165867.301560169] [PrecisionLandCustom]: Subscriber found, continuing
[DEBUG] [1761165867.301818408] [PrecisionLandCustom]: Publisher found, continuing
[DEBUG] [1761165867.401117460] [PrecisionLandCustom]: Got RegisterExtComponentReply
[DEBUG] [1761165867.416877529] [PrecisionLandCustom]: Arming check request (id=1, only printed once)
python3
QGround...
px4
python3
python3
python3
main
modifications
0 0
2
Pull Request #35

Precision Land on moving target

px4_roscon_workshop/precision_land/landing_on_moving_target.md

Quick Break

Coming Next: Precision Landing

Aerial Robotics Meetup, tonight!

Tonight, 8:00 to 11:00 PM
CRAFT Beer Market, 1 Adelaide St E (short walk from the venue)
Sponsored by Dronecode. **Food and drinks are on us.**

Talk shop on control architectures, state estimation, and open source flight stacks with the people who actually maintain them.

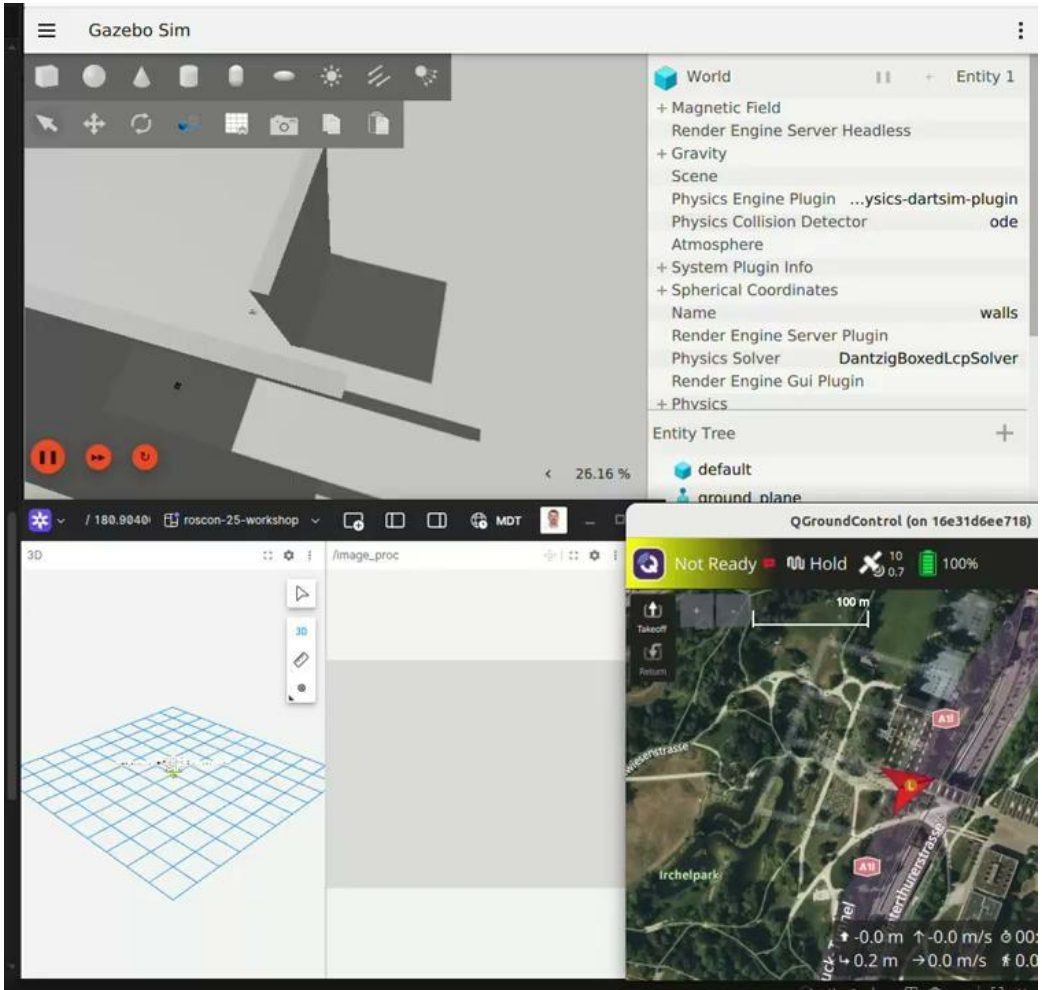


Teleoperation & Precision Land Exercise

Integrate all the concepts from today. Create a ROS2 package with a Mode Executor for **Precision Landing** and perform a precision landing in the maze we explored, using teleoperation. Incorporate the custom waypoints you recorded along the way (Create a CustomWaypoints Mode).

Hint: You can start by copying the **Custom Mode** package. Replace CustomYaw with PrecisionLand and add the waypoints you collected earlier.

Quick video


Gazebo Sim
World
Entity 1
Magnetic Field
Render Engine Server Headless
Gravity
Scene
Physics Engine Plugin ...ysics-dartsim-plugin
Physics Collision Detector ode
Atmosphere
System Plugin Info
Spherical Coordinates
Name walls
Render Engine Server Plugin
Physics Solver DantzigBoxedLcpSolver
Render Engine Gui Plugin
Physics
Entity Tree
default
ground_plane
26.16 %
roscon-25-workshop
MDT
3D
/image_proc
QGroundControl (on 16e31d6ee718)
Not Ready
Hold
100%
100 m
Takeoff
Return
Irchelpark
-0.0 m
-0.0 m/s
0.2 m
0.0 m/s

Quick Break

Coming Next:
SIM 2 Real, Q&A Resources & Show and Tell

Aerial Robotics Meetup, tonight!

Tonight, 8:00 to 11:00 PM
CRAFT Beer Market, 1 Adelaide St E (short walk from the venue)
Sponsored by Dronecode. **Food and drinks are on us.**

Talk shop on control architectures, state estimation, and open source flight stacks with the people who actually maintain them.



From simulation to real vehicles: Safety First

- Prop guards (indoor in particular)
- Geofence
- Extensive pre-flight checklist
- Always inspect the vehicle
- Configure the failsafes

SIM vs REAL

SIM

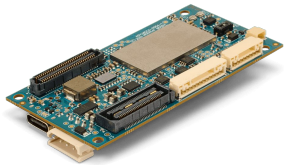
- Timing is of relative importance
- ROS 2 nodes run on high performance hosts
- No startup issues
- No PX4-ROS 2 comms issues
- MAVLink always available

REAL

- Timing is of GREAT importance
- ROS 2 nodes run on low/mid performance hosts
- Startup can be critical
- PX4-ROS 2 comms over serial can cause bandwidth issues
- MAVLink might not be available

Hardware Architecture

PX4 enabled flight controller + Companion computer



All in same chip





All in same package





Splitted architectures

PX4 - ROS 2 transport layer selection

	SERIAL	**UDP**
Support zenoh	NO	YES
Price	Cheaper	More expensive
Bandwidth	Limited	YES
MAVLink	Requires second serial	Included
Isolation	Intrinsic	Depends on network

Q&A, Resources & Hardware Show-and-Tell

- Open discussion and troubleshooting tips
- Review of key resources (links provided)
- Demo table with flight controllers, companion computers, and sensors

Open Lab & Individual Consultations

- One-on-one help with exercises
- Lightning talk

Resources

Community

- Meetings (dev call)
- Discord
- Forums
- Github (PX4, Dronecode, MAVLink, Pixhawk)
- Website
- Youtube
- Social media (linkedin, x)

Kahoot

Aerial Robotics Meetup, tonight!

Tonight, 8:00 to 11:00 PM
CRAFT Beer Market, 1 Adelaide St E (short walk from the venue)
Sponsored by Dronecode. **Food and drinks are on us.**

Talk shop on control architectures, state estimation, and open source flight stacks with the people who actually maintain them.

